

HTML5

Formulare in HTML

© Robert Schoblick, 09/2018

Hinweis:

Dieses Skript beinhaltet verschiedene Exkursionen in die (Server-)Skriptsprache PHP. Dies dient dem besseren Verständnis der gezeigten Beispiele. Diese Kurseinheit ist jedoch nicht als Grundlagenkurs zu PHP zu verstehen.

Inhalt

Formulare als Userschnittstelle	4
Das <form>-Element und entscheidende Attribute	4
Und „action“ – Was passiert mit den Eingaben?	5
Die Methoden GET und POST	6
GET	6
POST	7
Exkurs in PHP	8
Vorgabe der Codierung mit „enctype“	11
Formularelemente	12
Attribute „name“ und „value“	12
Das Universalelement <input>	13
Texteingaben	14
Eingaben mit Zeitbezug	17
Auswahlelemente und Auswahllisten	20
Eingabe von Zahlen und Spezialdaten	32
Versteckte Eingabe: type = „hidden“	39
Dateiupload	40
Auswahl einer Farbe mit type = „color“	44
Schaltflächen	46
Mehrzeilige Textfelder: Element <textarea>	52
Gruppierungen / „Fieldsets“	55
Das Element <fieldset>	55
„legend“ vs. „label“	56
Begrenzte Eingaben mithilfe von „Pattern“	57

Einfaches Beispiel: Rein numerische Eingabe mit 5 Stellen	57
Eingabe mit vier oder fünf Stellen	58
Gemischte Eingabe mit definiertem Format	59
Groß und Kleinschreibung ohne Ziffern.....	60
Leerzeichen akzeptieren	62
Direkter Textvergleich	63
Protokoll-Präfix vor Webadresse.....	64
Pflichteingaben	64

Formulare als Userschnittstelle

Formulare haben eine große Bedeutung im Webdesign. Von der einfachen Suchanfrage über ein Kontaktformular bis hin zu komplexen Online-Shop-Lösungen sind HTML-Formulare das Kernstück des Webauftritts. HTML generiert hierbei die Oberfläche, die mithilfe von CSS gestalterisch formatiert werden kann. Aktive Skripte (JavaScript auf der Client-Seite) oder PHP auf der Serverseite verarbeiten die über die Formularfelder getätigten Eingaben.

Das <form>-Element und entscheidende Attribute

HTML-Formulare werden von einem <form>-Container umschlossen. Die Attribute des <form>-Elementes legen fest, was nach dem Absenden des Formulars mit den Daten passiert und wie diese Daten an das verarbeitende Skript übergeben werden. Es ist möglich, selbstreferenzierende Formularskripte zu erstellen, allerdings muss dieses dann mit der Dateinamenerweiterung .php und mit entsprechenden auswertenden PHP-Skripten versehen werden.

Denkbar sind auch JavaScripte, mit denen die Eingaben verarbeitet werden. Die AJAX¹-Architektur begründet sich auf die Kombination von HTML-Formularen und JavaScript. Hierbei übertragen Skripte bereits während der Eingabe die Daten an den Server, ohne die komplette Webseite erneut laden zu müssen. In dieser Kurseinheit sollen AJAX und JavaScript zunächst nicht besprochen und stattdessen nur einfach PHP-Beispiele² besprochen werden.

Dem <form>-Element werden verschiedene Attribute übergeben. Die wichtigsten Attribute sind „action“ und „method“.

Hinweis:

¹ AJAX steht für **A**synchronous **J**avaScript and **X**ML. Hierbei kommunizieren Teilskripte einer Webseite mit dem Server und verändern den Inhalt der Seite entsprechend den Ergebnissen, ohne die Seite vollständig neu zu laden.

² In der Regel werden die im Formular eingetragenen Daten entweder sofort verarbeitet und das Ergebnis ausgegeben oder in einer Datenbank gespeichert. Auf das Thema Datenbanken wird in einem eigenen Modul der Softwareentwicklung eingegangen.

Wird das Element <form> ohne Attribute verwendet, gelten Default-Vorgaben:

- method="GET"
- action="eigener URL"
- encode="text/plain"

Und „action“ – Was passiert mit den Eingaben?

Mit dem Attribut „action“ wird die Adresse des Skriptes festgelegt, in dem die Formulardaten weiter verarbeitet werden. Wird dieses Attribut nicht verwendet, wird von einem selbstreferenzierenden Skript ausgegangen. Das bedeutet, dass sich das Formularskript selbst aufruft. Damit dies funktioniert müssen natürlich sowohl weiter verarbeitende Elemente als auch eine Prüfung vorgesehen werden, mit der festgestellt wird, ob bereits Formularinhalte vorhanden sind.

In der Regel wird explizit ein Skript definiert und die Adresse absolut oder relativ als Argument des Attributes übergeben. Beispiele hierfür sind:

Absolute Adressierung:

- action="http://www.weitergehts.at/weitergehts.php"
- action="192.168.99.1/weitergehts.php"

Bei der absoluten Adressierung wird das weiterverarbeitende Skript direkt unter der genannten Adresse aufgerufen. Es ist dabei nicht von Interesse, in welchem Verhältnis das Formular zum auswertenden Skript steht.

Relative Adressierung:

- action="weitergehts.php"
- action="../parallelfolder/weitergehts.php"
- action="./unterordner"/weitergehts.php"

Relative Adressen verweisen auf Ziele, die in einem genau definierten Zusammenhang zu dem aufrufenden Skript erreichbar sind. Im ersten der drei Beispiele befindet sich das Skript „weitergehts.php“ im gleichen Verzeichnis wie das aufrufende Skript.

Beim zweiten Beispiel wird die Adresse in einem Parallelverzeichnis gefunden. Dies wird durch die Einleitung mit der Sequenz „../“ deutlich. Die beiden führenden Punkte zwingen zum Sprung auf das nächst höhere Verzeichnis. Von

dort aus wird die Datei „weitergehts.php“ im genannten Pfad – hier /parallelfolder/ – gesucht.

Auch im dritten Beispiel wird die Datei „weitergehts.php“ in einem Unterverzeichnis gefunden. Dieses Unterverzeichnis ist aber aus der Ebene des aufrufenden Skriptes erreichbar. Die Sequenz „./“ (nur ein Punkt) steht für das aktuelle Verzeichnis.

*Die im <form>-Element mit dem Attribut „action“ übergebene Adresse wird nach dem Klick des „submit“-Buttons (siehe Seite **Fehler! Textmarke nicht definiert.**) aufgerufen und die Formulardaten mit der nachfolgend beschriebenen http-Methode übergeben.*

Die Methoden GET und POST

Das Hypertext Transfer Protocol (http³) definiert verschiedene „request methods“ (Anforderungsmethoden) beim Webserver⁴. Dies sind unter anderem die Methoden GET und POST.

GET

Die GET-Methode ist die Standard-Methode. Wird auf das Attribut „method“ im Element <form> verzichtet, wird automatisch *method=„GET“* eingestellt.

Mit der GET-Methode werden die Formulardaten direkt mit dem URI übertragen. Sie werden der Adresse durch ein Fragezeichen getrennt direkt nachgestellt. Um die Syntax muss sich der Webdesigner im Detail zwar nicht kümmern, jedoch ist es wichtig, die damit verbundenen Einschränkungen zu kennen:

- **Die Datenübertragung erfolgt im Klartext mit der Adresse.** Das bedeutet, dass alle mit dem Formular gesendeten Informationen aus der Adressleiste ablesbar sind, was insbesondere für die Übermittlung von

³ Bitte nicht verwechseln mit HTML! – http ist das Übertragungsprotokoll, mit dem u.a. auch Webseiten im TCP/IP-Netz übermittelt werden. HTML ist die Auszeichnungssprache, nach deren Angaben Webseiten im Browser aufgebaut werden.

⁴ Zur Erinnerung: Der Webbrowser sendet ein „Request“, eine Anforderung an den Webserver. Kann der Webserver diese Anforderung erfüllen, so sendet dieser wiederum die bestellte Webseite an den Browser. Während der Anforderung durch Übergabe eines URI (Uniform Resource Identifiers) können zusätzliche Parameter an den Server gesendet werden.

Zugangsdaten (sprich: Passwörter) ein eklatantes Sicherheitsrisiko darstellt.

- **Die Datenmenge ist limitiert.** Zwar wird die maximal mit der GET-Methode übertragbare Datenmenge vom verwendeten Webserver vorgegeben, jedoch sollte im Webdesign der Worst-Case angenommen werden. Während der Apache-Webserver sehr großzügig ist und durchaus bis zu 16kByte Datenmenge akzeptiert, sehen das andere Webserver deutlich enger. Ziemlich sicher lassen sich bis zu 255 Byte mit der GET-Methode senden, was allerdings bereits für einen einfachen E-Mail-Adresskopf sehr knapp bemessen ist.

```
<form method="GET">
  Vorname: <input type="text" name="vorname"><br>
  Nachname: <input type="text" name="nachname"><br>
  <button type="submit">Senden</button>
</form>
```

Das vorangehende HTML-Listing erzeugt ein einfaches – in Abbildung 1 – gezeigtes Formular. Es ruft sich nach dem Absenden selbst auf (das Attribut „action“ fehlt). Im gelb betonten Teil des URL ist im Klartext abzulesen, dass der Vorname „Max“ und der Nachname „Musterhuber“ übertragen wurde. Zu beachten ist bei dieser Methode unbedingt, dass auch Passwörter im Klartext im URL ablesbar sind.

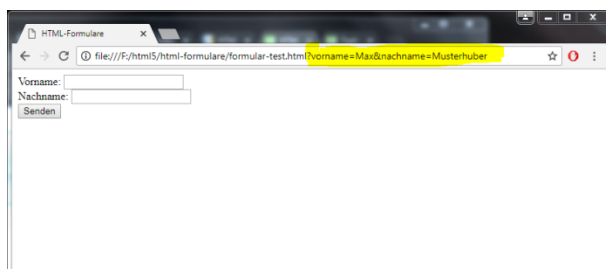


Abbildung 1: Beim Versand von Formular-Daten mit der GET-Methode werden die Daten der Adresse – getrennt durch das Fragezeichen – direkt und im Klartext nachgestellt.

POST

Die POST-Methode bietet gegenüber GET zwei Vorteile: Einerseits erfolgt keine Übertragung von Klartext-Daten direkt in der Adresszeile des Browsers und andererseits ist die übertragbare Datenmenge nahezu unlimitiert. So lassen sich auch Inhalte sehr komplexer Formulare und sogar binäre Dateien

(Programme, PDF, Audio, Video und Bilder etc.) an den Server senden. Dennoch sollte kein falscher Eindruck hinsichtlich der Übertragungssicherheit entstehen, denn die Daten werden mit der POST-Methode dem Header des http-Request angehängt und innerhalb diesem wiederum im Klartext versendet. Es erfordert allerdings ein entsprechendes Tool, um den http-Traffic mitzulesen. Der Blick „über die Schulter“ genügt also nicht mehr.

```
<form method="POST">
  Vorname: <input type="text" name="vorname"><br>
  Nachname: <input type="text" name="nachname"><br>
  <button type="submit">Senden</button>
</form>
```

Das vorangehende HTML-Listing zeigt eine Abwandlung des ersten Beispiels. Nach dem Absenden des Formulars wird allerdings ein anderes Bild im Browser erscheinen: Die Adressleiste enthält nach wie vor den URL, nicht aber mehr den in Abbildung 1 gelb unterlegten Teil.

Vorsicht!

Auch wenn mit der POST-Methode keine Daten im Klartext direkt in den URL geschrieben werden, bedeutet dies noch nicht, dass der Datenschutz gewährleistet ist. Das http überträgt nach wie vor alle Daten im Klartext! Um eine gewisse Vertraulichkeit zu erreichen (Passwörter und Mail-Inhalte etc. verschlüsselt), muss das Protokoll https⁵ gewählt werden.

Exkurs in PHP⁶

Der Webserver übernimmt die vom Client mit der http-Methode GET bzw. POST übertragenen Daten und speichert diese entsprechend in ein Array \$_GET[] bzw. \$_POST[]. Als Index für das Array wird das Attribut „name“ des jeweiligen „Input“-Elementes⁷ verwendet. Die Datei, muss auf dem Webserver

⁵ https steht für **H**ypertext **T**ransfer **P**rotocol **S**ecure. https wird über den Standard-Port 443 erreicht. Der Server muss ein SSL-Zertifikat bereitstellen, das von einer Certificate Authority (CA) auszustellen ist. Die Übertragung der Daten erfolgt in beide Richtungen verschlüsselt.

⁶ Um die mit PHP verbundenen Beispiele nachzuvollziehen, wird ein Webserver benötigt, der PHP unterstützt. Mit XAMPP kann eine geeignete Testumgebung auf dem eigenen PC installiert werden. Eine SQL-Datenbank ist für die Lektionen dieses Kurses nicht erforderlich.

⁷ Vgl. ab Seite 9.

die Dateinamenerweiterung „.php“ haben, damit der so genannte PHP-Parser aufgerufen werden kann. Eine rein als „html“ mit der Dateinamenerweiterung deklarierte Datei bewirkt dies nicht. Eventuell darin enthaltene PHP-Skripte würden nicht berücksichtigt werden.

Zur Erläuterung des Prinzips sei nun das Listing auf Seite 7 um einige PHP-Code-Zeilen⁸ erweitert (der hellgrau dargestellte Teil des Listings entspricht dem bereits gezeigten Code):

```
<?php
if ($_GET['vorname'] == "" AND $_GET['nachname'] == "")
{
    echo 'Es liegen noch keine Eingaben vor. <br>';
}
else {
    echo 'Folgende Eingaben liegen vor <br>';
    echo 'Vorname: ' . $_GET['vorname'] . ' <br>';
    echo 'Nachname: ' . $_GET['nachname'] . ' <br><br>';
}
?>

<form method="GET" action="form.php">
    Vorname: <input type="text" name="vorname"><br>
    Nachname: <input type="text" name="nachname"><br>
    <button type="submit">Senden</button>
</form>
```

Der in blauer Schrift dargestellte Code beschreibt das speziell für den Start des PHP-Parsers vorgesehene HTML-Element `<?php ... ?>`. Innerhalb dieses Containers gilt eine andere Syntax als sie in reinem HTML gültig ist. Dies betrifft auch Kommentarzeilen! HTML-Code kann durch „echo“-Kommandos (Darstellung in grüner Schrift) an den Browser gesendet werden. Der HTML-

⁸ PHP ist die „rekursive“ Abkürzung für PHP Hypertext Preprocessor. Es handelt sich um eine Programmiersprache. Sie wird nicht kompiliert, sondern direkt als Skript ausgeführt. PHP unterstützt objektorientierte Programmierung. Zur Ausführung von PHP muss auf dem Webserver ein PHP-Parser installiert und aktiviert sein.

Code muss dabei jedoch dem echo-Kommando als Argument übergeben werden. Die Argumente können in doppelten Hochkommata oder in einzelnen Hochkommata eingeschlossen werden. Das hier gezeigte Beispiel verwendet die Schreibweise mit einzelnen Hochkomma, da HTML-Code ebenfalls (Beispiel: Wertezuweisung in den Attributen eines HTML-Elementes) mit Doppelten Hochkommata arbeitet. Um Sonderzeichen, die in PHP selbst eine Bedeutung haben, in den Argumenten verwenden zu können, müssen diese mit einem vorangestellten Backslash „\“ maskiert werden.

Beispiel (hier das echo-Kommando mit Argumenten in doppelten Hochkommata):

```
echo "Hubert Musterhuber sagte: \"Hallo Welt!\" (Zitat Ende).";
```

Wird das Argument des echo-Kommandos in einzelne Hochkommata gepackt, ist auch diese Schreibweise möglich:

```
echo 'Hubert Musterhuber sagte: "Hallo Welt!" (Zitat Ende).';
```

Auf eine Maskierung kann hier verzichtet werden.

PHP ist eine Programmiersprache und kennt damit unter anderem auch Verzweigungen. In diesem Beispiel wurde die „if-Verzweigung“ (dargestellt in roter Schrift) und die Ergänzung „else“ verwendet. Als Bedingung prüft „if“, ob die Variablen-Arrays \$_GET[] (Darstellung in violetter Schrift) mit den Indizes „vorname“ und „nachname“ bereits mit einem Wert belegt sind. Beim ersten Aufruf der Seite ist dies natürlich noch nicht der Fall und es soll ein entsprechender Hinweis ausgegeben werden.

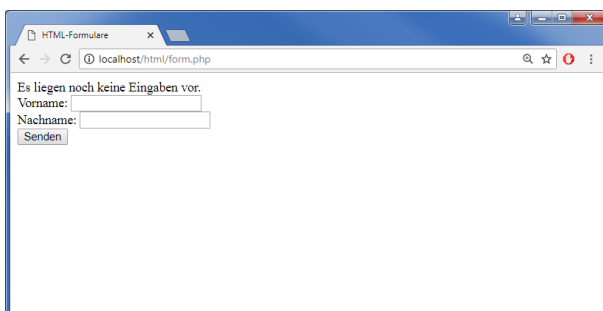


Abbildung 2: Der erste Aufruf zeigt die beiden leeren Eingabefelder des Formulars. Im darüber liegenden Bereich werden kein Vor- und kein Nachname angegeben. Ein Hinweis erklärt, dass noch keine Inhalte vorhanden sind.

Vorsicht Fehlerquelle!!! Wichtig ist bei der Betrachtung der Verzweigungsbedingung das zweifache Gleichheitszeichen! Dieses weist den

PHP-Parser an, die Werte zu vergleichen und auf Gleichheit zu prüfen. Wird dagegen nur ein Gleichheitszeichen geschrieben, so entspricht dies einer Zuweisung! Der PHP-Parser würde der Variablen den vermeintlichen Vergleichswert als neuen Inhalt zuweisen.

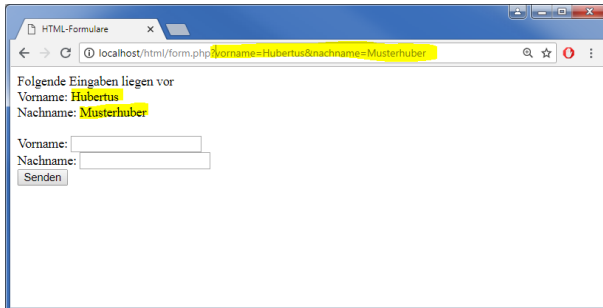


Abbildung 3: Der zweite Durchlauf nach der Eingabe gültiger Daten: Das kleine PHP-Skript gibt diese nun an der richtigen Position aus. Da es sich um die GET-Methode handelt, werden die Daten im Klartext im URL übergeben.

Das gleiche Skript kann – in etwas modifizierter Form – auch für Formulare eingesetzt werden, die mit der POST-Methode arbeiten. Das auszulesende Array heißt dann jedoch `$_POST[]`. Betroffen sind alle violett im Listing auf Seite 9 dargestellten Variablen. Alle anderen Bestandteile des Skriptes bleiben beim Gleichen.

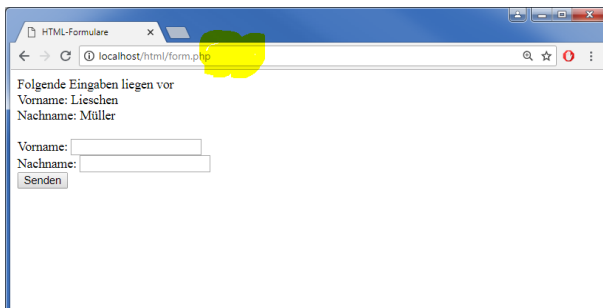


Abbildung 4: Das gleiche Formular wie bereits gesehen nach dem Absenden der Daten: Hier wurde jedoch mit der POST-Methode gearbeitet. Es werden dem URL keinerlei Daten beigefügt.

Vorgabe der Codierung mit „enctype“

Dieses Attribut ist unter Umständen sinnvoll, wenn die *POST-Methode* für das Formular verwendet wird. Es legt fest, ob und in welcher Weise die zu übertragenen Daten in ein URL-konformes Format umgewandelt werden sollen. Hierbei handelt es sich um eine Zeichendarstellung, die für die Verwendung als URL in der Adressleiste des Browsers zulässig ist.

Wird die GET-Methode verwendet, so erfolgt die Datenübertragung ohnehin als Anhang des URL, was eine entsprechende Konvertierung bereits erzwingt. Das „enctype“-Attribut ist deswegen bei der GET-Methode nicht erforderlich.

Attribut-Wert	Bedeutung
text/plain	Reine Text-Übermittlung, lediglich Leerzeichen werden in ein „+“ konvertiert. Alle übrigen Zeichen bleiben wie geschrieben.
multipart/form-data	Dieser Typ wird gewählt, wenn binäre Daten übertragen werden sollen (z.B.: Datei-Upload). Es erfolgt keine Konvertierung.
application/x-www-form-urlencoded	Es werden alle Zeichen in eine URL-taugliche Form konvertiert. Sonderzeichen werden durch ihren (hexadezimalen) ASCII-Code dargestellt.

Wird auf das Attribut „enctype“ verzichtet, so wird als Default-Wert „text/plain“ angenommen.

Formularelemente

Formulare bieten auf der Browser-Oberfläche Eingabefelder an. Diese werden in HTML von <input>-Elementen erzeugt. Die hier eingetragenen Daten werden nach Absenden (Klick auf den „Submit“-Button, vgl. ab Seite **Fehler! Textmarke nicht definiert.**) an das weiter verarbeitende Skript gesendet. Die Adresse dieses Skriptes gibt das Attribut „action“ im Element <form> vor (siehe Seite 5).

Die Eingabemöglichkeiten sind sehr vielseitig, was sich in der Vielzahl der Typen widerspiegelt. Bevor diese Typen jedoch im Detail erläutert werden, soll ein kurzer Einblick in den Transport der Formulardaten gegeben werden.

Attribute „name“ und „value“

Das HTML-Formular sendet grundsätzlich Datenpaare an das weiter verarbeitende Skript. Diese Datenpaare bestehen aus einem Bezeichner für das

betreffende Eingabefeld. Dieser Bezeichner wird mit dem Attribut „name“ festgelegt. Zur Erinnerung sei auf das Skript auf Seite 9 verwiesen. Innerhalb des <form>-Containers befindet sich unter anderem die folgende HTML-Zeile:

```
Vorname: <input type="text" name="vorname">
```

Dem Eingabefeld wurde das Attribut name=„vorname“ zugewiesen. „vorname“ ist damit als *Bezeichner* Teil des mit diesem Element erzeugten Datenpaares.

Der in das Formularfeld vom Besucher der Seite eingegebene Inhalt wird gemeinsam mit diesem Bezeichner an den Server übertragen und dort von einem Skript ausgewertet⁹.

Im gleichen Skript ist innerhalb des PHP-Blocks die folgende Zeile zu finden:

```
echo 'Vorname: ' . $_GET['vorname'] . ' <br>';
```

Das Formular hat in diesem Beispiel die Formulardaten mit der GET-Methode übertragen. Das zuvor im <input>-Element verwendete Attribut name=„vorname“ gibt hier den Bezeichner für das Index des \$_GET[]-Arrays vor, dessen Inhalt nun die übergebenen Daten sind. Diese lassen sich vom Empfängerskript auslesen und weiter verarbeiten bzw. in eine Datenbank speichern.

Das Universalelement <input>

Beinahe alle Eingabe-Felder eines HTML-Formulars werden mit dem Element <input> erzeugt. Man kann somit quasi von einem Formular-Universalelement sprechen. Die einzige Ausnahme stellt hier <textarea>¹⁰ für ein mehrzeiliges Eingabeelement in HTML-Formularen dar.

Die Art und Gestalt der HTML-Formularfelder wird durch den jeweiligen Typ festgelegt, der mit dem Attribut „type“ definiert wird. Die verschiedenen Formularfeld-Typen sollen nun im Detail besprochen werden.

⁹ An dieser Stelle seien die AJAX-Technologien außer Acht gelassen. Sie ermöglichen eine direkte Auswertung der Eingaben ohne die Webseite erneut zu laden. Somit können die Daten bereits vor Absenden des Formulars auf der Serverseite analysiert werden. In diesem Kurs seien jedoch nur die reinen HTML/CSS-Möglichkeiten betrachtet.

¹⁰ Vgl. ab Seite 19.

Texteingaben

Texte sind die prominentesten Daten, die in ein Formular eingetragen werden können. Grundsätzlich zählen auch Zahleneingaben, Web- und E-Mail-Adressen dazu, die hier jedoch in eigenen Bereichen diskutiert werden.

Texte werden für Beiträge in Blogs, für Suchanfragen, das Ausfüllen von Kontaktformularen oder aber für die Eingabe von Zugangsdaten verwendet.

type = „text“

Der Standard-Eingabetyp des `<input>`-Elementes ist „text“. Wird kein „type“-Attribut gesetzt, wird automatisch dieser Typ angenommen. Diese Eingabeelemente erzeugen ein einzeliges Eingabefeld, dessen Breite mit dem Attribute „size“ festgelegt werden kann (vgl. Seite **Fehler! Textmarke nicht definiert.**). Doch Vorsicht: Dieses Attribut ist einem CSS-Format mit der Eigenschaft „width“ untergeordnet.

Der nachfolgende Code-Ausschnitt ist bereits aus dem Eingangsbeispiel bekannt. Abbildung 1 zeigt, wie das Feld in der Praxis erscheint.

Vorname: `<input type="text" name="vorname">`

Mit dem Attribut „placeholder“ kann ein beschreibender Text in das Formularfeld eingesetzt werden, der jedoch beim Versenden des Formulars *nicht Inhalt des Datensatzes* wird.

`<input type="text" name="wahl" placeholder="Platzhalter">`

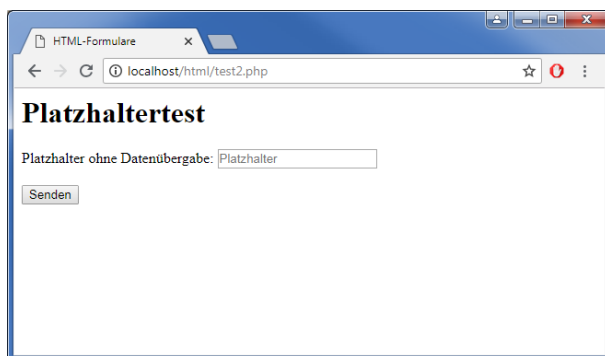


Abbildung 5: Der Text innerhalb des Eingabefeldes ist bereits mit dem Öffnen des Formulars vorbelegt. Beim genauen Hinschauen fällt auf, dass dieser in grauer Schrift eher defensiv erscheint. Es handelt sich um einen Platzhalter, mit dem erklärende Texte in das Feld eingesetzt werden können. Beim Absenden des Formulars wird dieser Text jedoch nicht als Datum übertragen.

Alternativ zu „placeholder“ kann das Attribut „value“ verwendet werden. Auch mit diesem Attribut wird das Eingabefeld vorbelegt, jedoch handelt es sich hier

um einen echten Eintrag, der beim Absenden an das weiter verarbeitende Skript übertragen wird. Der Besucher der Seite hat allerdings die Möglichkeit, diesen Eingabevorschlag durch Überschreiben zu verändern. Die gleichzeitige Verwendung der Attribute „placeholder“ und „value“ ist nicht sinnvoll, weil „value“ dominiert und der Platzhalter nicht im Feld angezeigt wird.

```
<input type="text" name="wahl" value="Wert">
```

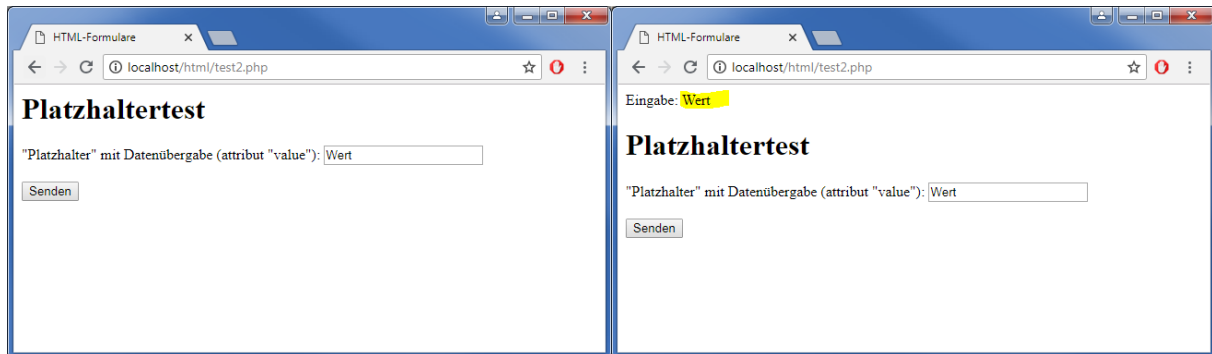


Abbildung 6: Mit dem Attribut "value" wird nicht nur ein Text in das Eingabefeld vorbelegt, sondern zugleich auch ein Standard-Wert definiert, der - wenn er nicht durch den Besucher der Seite im Formular verändert wird - als Eingabedatum an das verarbeitende Skript übertragen wird.

type = „search“

Beim Typ „search“ handelt es sich im Grunde genommen auch nur um ein einfaches Text-Eingabefeld. HTML5 folgt jedoch konsequent dem Pfad, oft genutzte und meist mit individuellen Klassen gekennzeichnete Elemente einer semantischen Struktur anzupassen und damit das Programmierverhalten zu vereinheitlichen. Dies geschieht mit der speziellen Typisierung des Suchfeldes.

```
<input type="search" name="suchfeld">
```

Die Differenzierung mithilfe des für HTML definierten Attributes erlaubt eine einfache Formatierung mit CSS ohne zusätzliche Klassendeklaration. Beispielsweise kann mithilfe des *CSS-Attribute-Selektors* das Suchfeld in besonderer Weise formatiert werden. Das folgende CSS-Beispiel zeigt die Umrahmung des Suchfeldes mit einem grünen Rahmen mit abgerundeten Ecken.

```
<style>
```

```
[type=search]
```

```
{
```

```
border: 2px solid green;
```

```
border-radius: 5px;
}
```

```
</style>
```

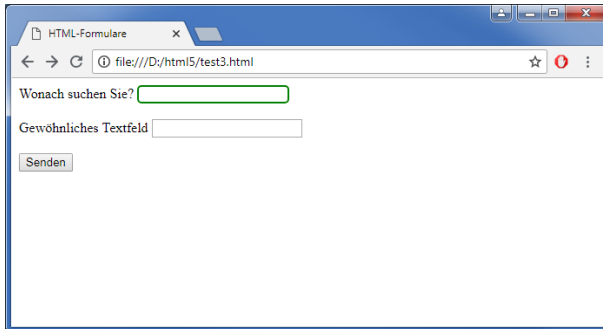


Abbildung 7: Das Suchfeld hebt sich nach CSS-Formatierung von den übrigen Text-Eingabefeldern ab.

type = „password“

Ein besonderes Text-Eingabefeld stellt der Typ „password“ dar. Auch dieses Formularfeld erfasst einzeilig einen Text, verdeckt jedoch die eingegebenen Zeichen. Es erfolgt also eine „blinde Eingabe“.

```
<input type="password" name="password">
```

Doch Vorsicht: Die Verwendung des <input>-Elementes ist keine Garantie für Datensicherheit! Es erfolgt lediglich eine verdeckte Eingabe. Die Sicherheit bei der Übermittlung zum Server hängt von der Verwendeten Methode und vom Übertragungsprotokoll (http/https) ab. Nur https ist verschlüsselt, stellt allerdings an den Webserver zusätzliche Anforderungen.

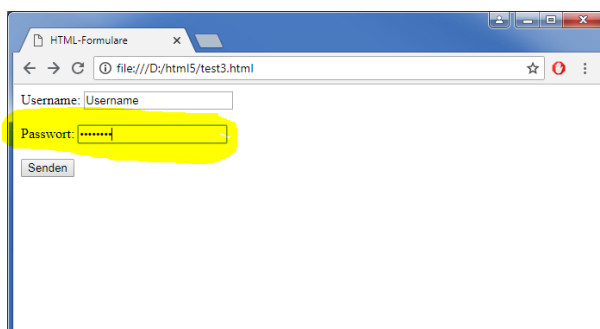


Abbildung 8: Gelb hervorgehoben ist hier das Eingabefeld für ein Passwort. Die Eingabe selbst wird durch Bullets versteckt.

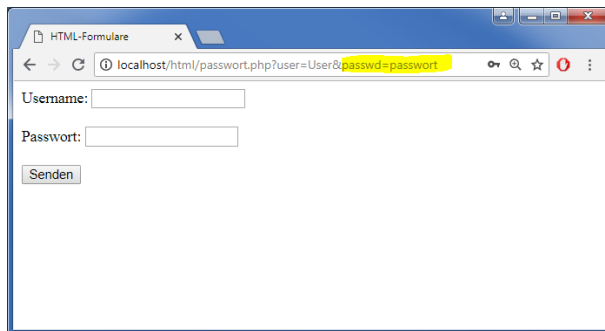


Abbildung 9: Hier ist auch die verdeckte Eingabe eines Passwortes witzlos! Wird das Formular mit der GET-Methode gesendet, so erscheint das Passwort im Klartext in der Adressleiste. Auch die POST-Methode ist kein zuverlässiger Schutz, denn auch wenn keine Klartextausgabe im Browser erfolgt, erfolgt doch die Übermittlung der Daten im Klartext und kann mit einfachen Sniffer-Programmen „mitgelesen“ werden. Nur die POST-Methode UND die verschlüsselte Übertragung (https) bieten eine gewisse Sicherheit.

Eingaben mit Zeitbezug

Viele Formulare fordern zeitbezogene Eingaben. Das kann zum Beispiel ein Geburtsdatum oder die Definition des Geschäftsjahres, einer Zahlungsfrist und vieles mehr sein. Natürlich sind auch derartige Eingaben in Textfeldern möglich, aber die Verarbeitung der Eingaben – beispielsweise in Datenbanken – erfordert dann zusätzlichen Aufwand für die Prüfung und die Anpassung an die geforderten Formate. Spezielle Feldtypen erleichtern sowohl dem Webdesigner als auch den Besucher der Seite den Umgang mit dem Formular.

type = „date“

Ein Datum lässt sich sehr komfortabel mit einem HTML-Formular erfassen, wobei das Format bereits so aufbereitet wird, dass die Eingabe einfach mit einem Skript ausgewertet und verarbeitet werden kann. Allerdings muss beim Speichern in eine SQL-Datenbank etwas aufgepasst werden, weil Datumsformate im internationalen Vergleich sehr unterschiedlich definiert sind und somit die Reihenfolge von Tag, Monat und Jahr sowie die Anzahl der Stellen und die Form der Separatoren möglicherweise anzupassen sind.

Die folgende HTML-Code-Zeile baut ein Datums-Eingabefeld in das Formular ein. Dieses kann direkt ausgefüllt werden. Alternativ dazu lässt sich ein Datums-Editor mit einem Klick auf das Pfeilsymbol öffnen und das gewünschte Datum aus einem Kalender mit der Maus wählen.

```
<input type="date" name="datum"><br>
```

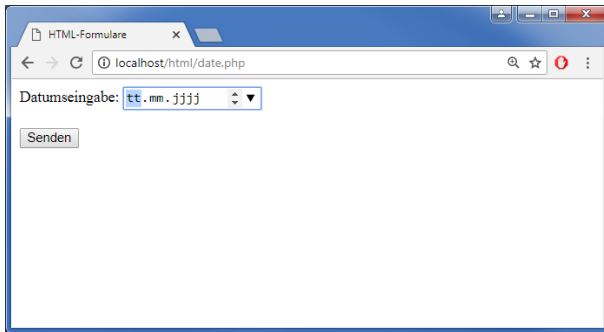


Abbildung 10: Mit dem Typ "date" gibt das Eingabefeld bereits ein gültiges Datumsformat vor. Das geforderte Eingabeformat ist eindeutig und ungültige Werte werden korrigiert (Tage nur 1 bis 31, Monate nur 1 bis 12).

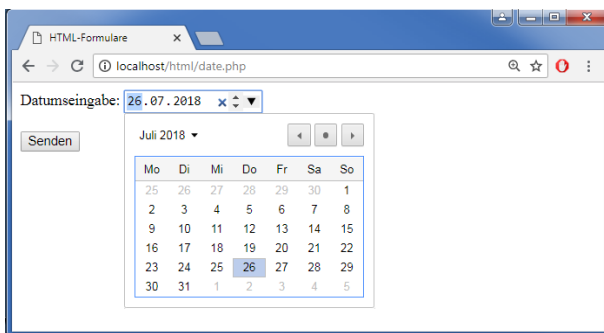


Abbildung 11: Auf aktuellen Browsern wird bei Bedarf auch ein Kalender eingeblendet, mit dessen Hilfe das Datumsfeld per mausklick ausgefüllt werden kann.

type = „datetime-local“

Ähnlich wie das Eingabefeld vom Type "date" kommt "datetime-local" daher. Allerdings wird das Eingabefeld zusätzlich um die Uhrzeit ergänzt. Es wird sowohl für das Datum – dieses kann auch hier wieder per Mausklick über einen Kalender gewählt werden – als auch für die Uhrzeit ein gültiger Wert erwartet.

```
<input type="datetime-local" name="datumZeit">
```

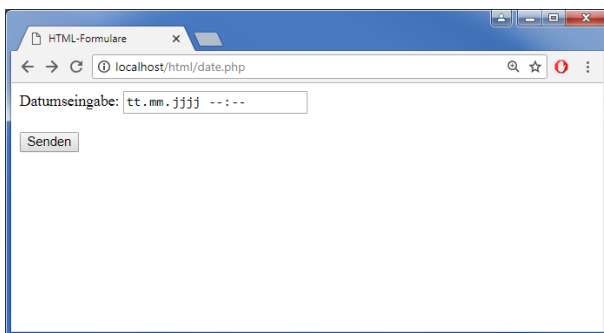


Abbildung 12: Der Typ "datetime-local" bietet in einem Feld die Eingabemöglichkeit für ein Datum und die Uhrzeit.

type = „week“

Eine weitere Abwandlung des Eingabefeldes vom Typ „date“ ist „week“. Hier wird allerdings nicht das Datum exakt auf den Tag genau verlangt, sondern die Kombination aus Kalenderwoche und Jahr. Für die vereinfachte Eingabe ist auch hier wieder die direkte Eingabe über die Tastatur als auch die Wahl per Mausklick im Kalender möglich.

```
<input type="week" name="woche">
```

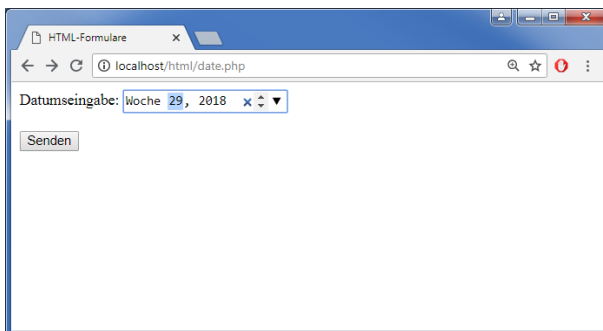
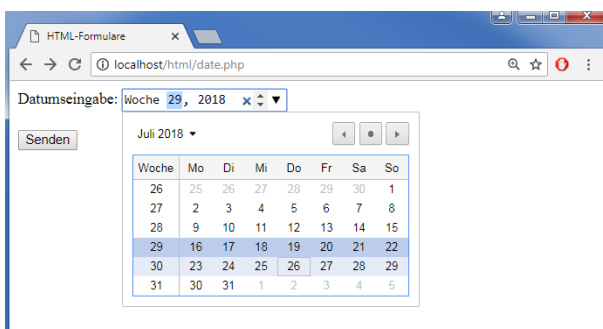


Abbildung 13: Termin-Ankündigungen werden gerne für "Kalenderwochen" definiert. Auch hierfür gibt es einen Typ für ein HTML-Formularfeld.



Woche	Mo	Di	Mi	Do	Fr	Sa	So
26	25	26	27	28	29	30	1
27	2	3	4	5	6	7	8
28	9	10	11	12	13	14	15
29	16	17	18	19	20	21	22
30	23	24	25	26	27	28	29
31	30	31	1	2	3	4	5

Abbildung 14: Das Eingabefeld vom Typ "week" kann per Mausklick auch über einen Kalender ausgefüllt werden.

type = „month“

Ähnlich wie beim zuvor gezeigten Typ „week“ kann die Eingabe auf einen Monat und ein bestimmtes Jahr beschränkt werden. Die Eingaben sind wie bereits gesehen sowohl direkt als auch per Mausklick möglich.

```
<input type="month" name="monat">
```

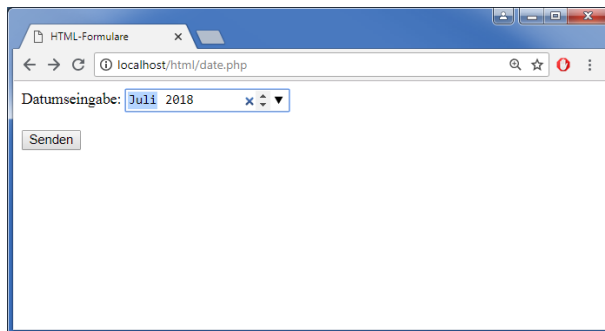


Abbildung 15: Der Typ "month" ermöglicht die Eingabe von Monat und Jahr. Auch hier kann die Auswahl wieder per Mausklick in einem Kalenderfenster getroffen werden.

type = „time“

Eine reine Zeiteingabe ist mit dem Typ „time“ möglich. Hier wird ein Feld zur Eingabe von Stunden und Minuten, getrennt durch einen bereits vorgegebenen Separator, in das Formular eingefügt. Die Eingabe kann direkt über die Tastatur oder mithilfe der Pfeiltasten erfolgen.

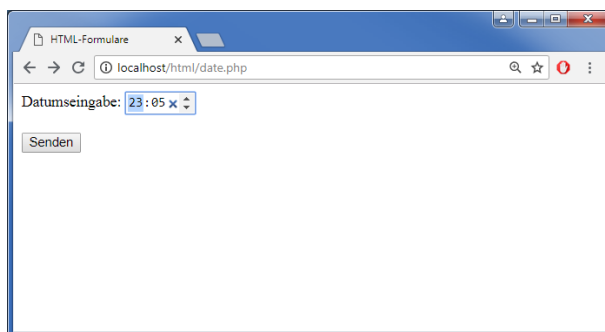


Abbildung 16: Wird die Eingabe einer reinen Uhrzeit gewünscht, so ist dies mit dem Typ "time" möglich.

Auswahlelemente und Auswahllisten

Wo immer es möglich ist, erleichtern Formulareingaben mit exakt definierten Daten die weitere Verarbeitung und vermeiden Dubletten in Datenbanken. Neben diesen Vorzügen erweisen sich feste Auswahlelemente und –Listen auch für den Besucher der Seite als komfortabler gegenüber einer Freitexteingabe. Das Formular kann schnell und bequem mit der Maus oder einem Fingertipp auf den Touchscreen ausgefüllt werden. Es gibt verschiedene Alternativen:

- Radio-Button: Eine einzige Auswahl in einer Angebotsgruppe
- Checkbox: Eine oder mehrere Wahlmöglichkeiten in einer Angebotsgruppe
- Auswahllisten

type = „radio“

Die so genannten „Radio-Button“ sind eine Gruppe von Eingabefeldern mit dem Typ „radio“. Sie erfordern nicht nur zur Referenzierung der übergebenen Daten im GET/POST-Array ein Attribut „name“, sondern verlangen überdies auch noch, dass der Name bei allen Radio-Button, die einer Gruppe angehören, gleich ist. Damit wird eine wichtige Eigenschaft der Radio-Button verständlich, mit der sich diese von den „Checkboxes“ unterscheiden (vgl. ab Seite 25): Es kann nur ein einziges Feld der Gruppe aktiviert werden.

Bereits mit der Präsentation des Formulars kann jedoch mit dem Attribut „checked“ eine Auswahl vorselektiert werden. Diese wird mit einem Klick auf einen anderen Radio-Button wieder deaktiviert. Allerdings wird auf diese Weise sichergestellt, dass in jedem Fall eine Wahl getroffen und kein leeres Element abgeschickt wird.

Um dem weiter verarbeitenden Skript einen Wert übergeben zu können, wird jedem <input>-Element ein Wert mit dem Attribut „value“ zugewiesen. Es erfolgt also keine interaktive Eingabe durch den Seitenbesucher. Dieser wählt lediglich eine von mehreren Alternativen aus. Auf der Seite des Empfängerskriptes hat dies den Vorteil, in jedem Fall klar definierte Werte¹¹ angeboten zu bekommen.

Das folgende Beispiel¹² zeigt ein Formular mit einer Liste aus sieben Radio-Button-Eingabefeldern. Das Formular sendet die Daten „an sich selbst“, weshalb es eingangs in einem kleinen PHP-Skript¹³ prüft, ob bereits Daten vorhanden sind (blaue Schrift). Es wurde die „POST“-Methode gewählt (vgl. ab Seite 7).

```
<?php
if ($_POST['wahl'])
{
    echo 'Folgende Auswahl wurde getroffen: ' . $_POST['wahl'];
```

¹¹ Dies ist auch bei Checkboxes und Auswahllisten der Fall.

¹² Aus Gründen der Übersicht ist das Formular etwas gekürzt dargestellt worden. Es kann in eigenen Versuchen beliebig um weitere <input>-Elemente ergänzt werden.

¹³ Da der PHP-Parser nur von einer PHP-Datei aufgerufen werden kann, muss die Dateinamenerweiterung .php lauten. Eine Datei mit der Endung .htm oder .html funktioniert nicht!

}

?>

Der folgende Abschnitt des Formulars ist wieder reiner HTML-Code. Das Formular wird wie bereits gelernt in ein Container-Element `<form>` eingefasst. Als Verarbeitungsmethode wird „POST“ gewählt. Da ein „action“-Attribut fehlt, kann daraus abgeleitet werden, dass sich das Formular die eigene Seite selbst aufruft und die Daten an „sich selbst“ schicken wird (vgl. ab Seite 5).

In der farblich dargestellten Code-Zeile ist eines der sieben Radio-Button definiert. Wie fast alle Formular-Eingaben wird auch der Radio-Button mit einem `<input>`-Element realisiert. Der Type „radio“ legt die Darstellung als Radio-Button fest. Um die Eingaben auswerten zu können, bedarf es selbstverständlich eines Wertepaares, welches aus dem Attribut „name“ (entspricht der Variablen oder exakter dem Index innerhalb des Arrays, in das die Information gespeichert wird) und dem Attribut „value“, dem eigentlichen Wert, der übergeben wird. Es fällt auf, dass alle Radio-Button den gleichen Namen besitzen (grün dargestellt). Sie unterscheiden sich lediglich in ihrem Wert (braune Schrift).

```
<form method="POST">
```

```
<h1>Wohin geht's in den Urlaub?</h1>
```

```
<p><input type="radio" name="wahl" value="USA"> USA</p>
```

```
... <!-- Weitere Formularzeilen hier einsetzen! -->
```

```
<p><input type="radio" name="wahl" value="Balkonien">
Balkonien</p>
```

```
<br>
```

```
<button type="submit">Senden</button>
```

```
</form>
```



Abbildung 17: In einem HTML-Formular mit Radiobutton kann nur ein Button aktiviert werden, denn alle Input-Felder haben den gleichen Namen.



Abbildung 18: Welcher Radio-Button aktiviert wurde, wird dem auswertenden Skript mithilfe des Attributes "value" mitgeteilt.

Das eben gezeigte Skript funktioniert und ist ohne Weiteres auch praxistauglich. Allerdings weist es eine unschöne Eigenschaft: Um eine Auswahl zu treffen, muss man zwingend direkt auf den Radio-Button klicken. Besonders, wenn das Formular auf einem Smartphone mit vergleichsweise kleinem Viewport bearbeitet werden soll, erweist sich dies als unkomfortabel. Man darf heute – zu recht – erwarten, dass die Auswahl auch mit einem Klick auf den Beschreibungstext aktiviert wird. Dazu wird ein – für Formularfelder sehr nützliches – zusätzliches Element verwendet: `<label>`.

Wie es der Name des Elementes vermuten lässt, bezeichnet `<label>` die Beschriftung des Eingabefeldes. Da es sich nicht um ein Container-Element handelt, muss ein Bezug zwischen dem `<input>`- und dem `<label>`-Element hergestellt werden. Das Attribut „name“ ist insbesondere bei einem Radio-

Button-Formular ungeeignet, weil alle Elemente einer Gruppe den gleichen Namen führen. Hier kommt das Attribut „id“ zum Einsatz, das in das <input>-Feld gesetzt wird. Eine ID darf innerhalb eines HTML-Dokumentes nur ein einziges Mal vergeben werden. Damit ist eine eindeutige Zuordnung gegeben.

Innerhalb des <label>-Elementes verweist das Attribut „for“ auf die ID des zugehörigen Eingabeelementes¹⁴.

```
<input type="radio" name="wahl" value="USA" id="button1">
<label for="button1"> USA<br></label>
```

Es geht auch etwas eleganter:

```
<p><input type="radio" name="wahl" value="USA" id="button1">
<label for="button1"> USA</label></p>
```

Zu beachten ist, dass es sich sowohl bei <input> als auch bei <label> um so genannte „Inline-Elemente“ handelt. Diese Elemente bewirken also keinen Zeilenumbruch und so können Radio-Button auch nebeneinander erscheinen.

Sollen die einzelnen Wahlmöglichkeiten untereinander präsentiert werden, so sind Zeilenumbrüche zu setzen. Hier gibt es verschiedene Alternativen:

- Deklaration eines der Elemente mit CSS als „Block“
- Verwendung eines „harten“ Zeilenumbruches

- Einfassung jedes <input>-<label>-Paares in einem Absatz-Container <p>...</p>

¹⁴ Alternativ zur Verwendung einer ID und der Referenzierung mit dem Attribut „for“ im <label>-Element kann das <label>-Element auch als Container für das betreffende Formular-Element verwendet werden und dieses in dessen Inhalt umschließen.

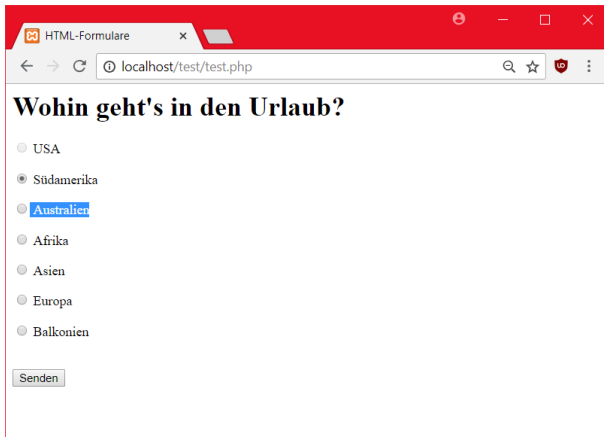


Abbildung 19: Problem beim Verzicht auf das Attribut "label": Um eine Auswahl treffen zu können, muss der Radio-Button direkt angeklickt werden. Ein Klick auf die Beschreibung bleibt ohne Wirkung.



Abbildung 20: Die Elemente `<input>` und `<label>` sind Inline-Elemente. Ihr Einsatz bewirkt keinen Zeilenumbruch.

type = „checkbox“

Ähnlich wie Radio-Button wird der HTML-Code für Checkboxes geschrieben. In der Tat zeigen die beiden Formularelemente Parallelen, unterscheiden sich jedoch stark in ihrer Wirkung:

Elemente für Radio-Button einer Gruppe werden mit dem gleichen Namen-Attribut belegt. Auch bei Checkboxes ist dies möglich. Hier *kann* der Name für alle Elemente gleich sein (muss es aber nicht). Allerdings steckt hier der Teufel im Detail, denn wenn eine Mehrfachauswahl möglich sein soll, müssen diese Namen intern auf ein Array verweisen. Das passiert mit dem Anhang von (leeren) eckigen Klammern. Ein Beispiel:

Das Attribut (eines `<input>`-Elementes)

```
name="wahl[]"
```

wird nach Übertragung mit der POST-Methode in das folgende Array(!) überführt:

```
$_POST['wahl']
```

Bei der Programmierung des auswertenden Skriptes muss dies beachtet werden!

Beispielsweise kann das folgende kleine PHP-Skript¹⁵ feststellen, ob Daten bereits über ein Formular (hier: mit der POST-Methode) übergeben wurde und diese der Reihe nach auslesen:

```
<?php
if ($_POST['wahl'])
{
    foreach ($_POST['wahl'] as $auswahl)
        echo 'Folgende Auswahl wurde getroffen: ' . $auswahl . '<br>';
}
?>
```

Die „foreach“-Schleife wird solange durchlaufen, wie es Daten im Array \$_POST['wahl'] gibt. Diese werden dann direkt in einer „echo“-Anweisung ausgegeben. Die „if“-Verzweigung prüft hingegen, ob Daten im Array vorliegen. Ist dies nicht der Fall wird die Ausgabe übersprungen¹⁶.

Das folgende reine HTML-Skript zeigt ein Formular mit Checkboxes, das von dem zuvor beschriebenen PHP-Skript ausgewertet werden kann. Da auch hier kein „action“-Argument im <form>-Element gesetzt wurde, wird das PHP-Skript in die gleiche Datei (Dateinamenerweiterung muss .php sein) geschrieben. Mit dem Klick auf den Absenden-Button ruft sich die Datei quasi selbst auf.

¹⁵ Hinweis: Dem aufmerksamen Leser wird auffallen, dass der Code innerhalb der „foreach“-Schleife nicht in geschweiften Klammern steht. Auch wenn hier aus platztechnischen Gründen der Code zwei Zeilen belegt, handelt es sich lediglich um eine einzige PHP-Anweisung (echo ...). In diesem Fall kann auf das Setzen von Klammern verzichtet werden.

¹⁶ Alternativ kann hier die PHP-Funktion isset(\$argument) verwendet werden. Sie liefert logisch „wahr“, wenn das Argument einen Inhalt besitzt.

Bei der Betrachtung der farblichen Stellen im Skript fällt die markante Ähnlichkeit zum zuvor beschriebenen Skript für ein „Radio-Button“-Formular auf. Allerdings unterscheidet sich der Code nicht nur im Argument „type“ und in der Ergänzung der eckigen Klammern beim Argument „name“, sondern auch im Ergebnis im Browser: Während nur ein Radio-Button aktiv sein kann, kann hier von keiner Auswahl (alle Checkboxes sind deaktiviert) über eine Checkbox bis hin zu alle Checkboxes markiert sein

```
<form method="POST">
<h1>Ausstattungskonfiguration</h1>

  <p><input      type="checkbox"      name="wahl[]"      value="radio"
id="button1">

    <label          for="button1">          Soundsystem
(CD/USB/Radio)</label></p>

  <p><input      type="checkbox"      name="wahl[]"      value="navi"
id="button2">

    <label for="button2"> Navigationssystem</label></p>

  <p><input      type="checkbox"      name="wahl[]"      value="GPS"
id="button3">

    <label for="button3"> GPS-Ortung</label></p>

  <br>

  <button type="submit">Senden</button>
</form>
```

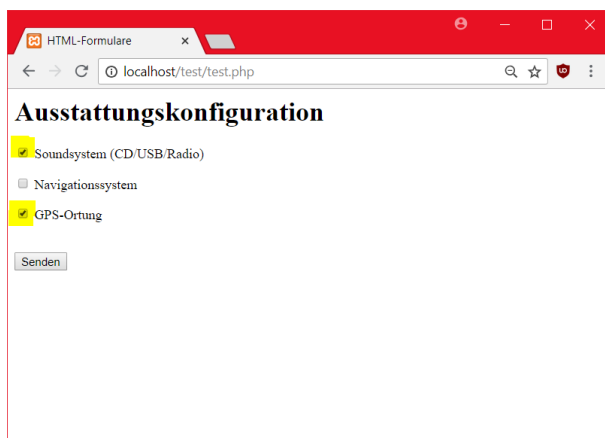


Abbildung 21: Optisch ist der Unterschied zu den Radio-Button gering: Checkboxes kommen in rechteckigen Feldern daher. Funktional ist der Unterschied aber groß, denn Checkboxes erlauben auch eine mehrfache Auswahl.

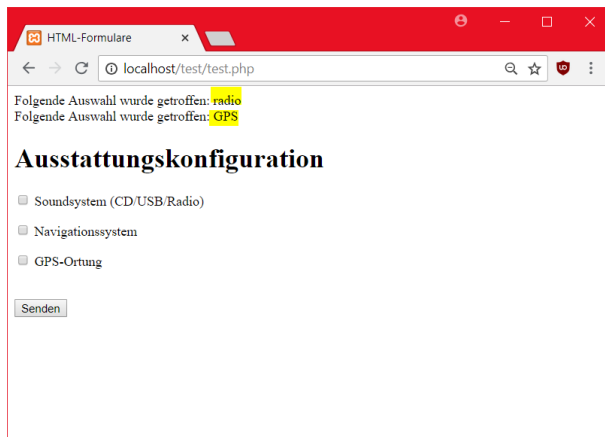


Abbildung 22: Nach dem sequentiellen Auslesen des `$_POST`-Arrays mithilfe der `foreach`-Schleife werden die zuvor übergebenen Daten im Browser angezeigt.

Wer auf die Benutzung eines Arrays verzichten möchte, kann jeder Checkbox einen eigenen Namen zuweisen. Auch dies erlaubt das gezielte Auswerten der Formulardaten. Ebenso können natürlich auch Radio-Button mit jeweils eigenen Namen in einem Formular platziert und damit „mehrfache“ Auswahlen ermöglicht werden.

Einfache Auswahlliste

Bei der Auswahlliste wird das Element `<input>` nicht verwendet! Neben der `<textarea>` ist dies die einzige Ausnahme, wo `<input>` nicht für die Darstellung der Eingabefelder zuständig ist.

Für die Gestaltung von Auswahllisten werden zwei neue HTML-Elemente benötigt:

`<select> ... </select>` ist der Container für die gesamte Auswahlliste. Hier wird mit dem bereits bekannten Attribut „name“ der Bezeichner für das Index im späteren `$_POST`- bzw. `$_GET`-Array definiert. Der Wert wird allerdings diesmal mit dem „value“-Argument in einem anderen Element `<option>` zugewiesen¹⁷.

Für jeden einzelnen Listeneintrag wird nun ein eigenes `<option>`-Element geschrieben.

¹⁷ Wird kein Argument „value“ im Element `<option>` gesetzt, so wird der Inhalt des Elements übergeben. Das ist nicht immer erwünscht, weil für den User sichtbar eine gute selbsterklärende Beschreibung der Auswahl wichtig ist, wogegen für die verarbeitenden Skripte klare und einfache Parameter zu empfehlen sind, die während der Programmierung mit geringer Fehlerwahrscheinlichkeit in den Code geschrieben werden können.

```

<form>
  <label for="selektion">Wählen Sie die Konfiguration</label>
  <select name="wahl[]" id="selektion">
    <option value="">Bitte wählen ...</option>
    <option value="radio">Radio/CD/USB</option>
    <option value="navi">Navigation</option>
    <option value="gps">GPS-Ortung</option>
  </select>
  <br>
  <button type="submit">Senden</button>
</form>

```

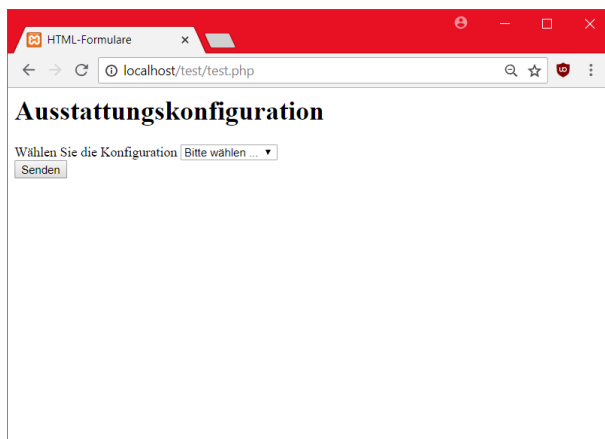


Abbildung 23: Das Ergebnis beim ersten Aufruf des Formulars.

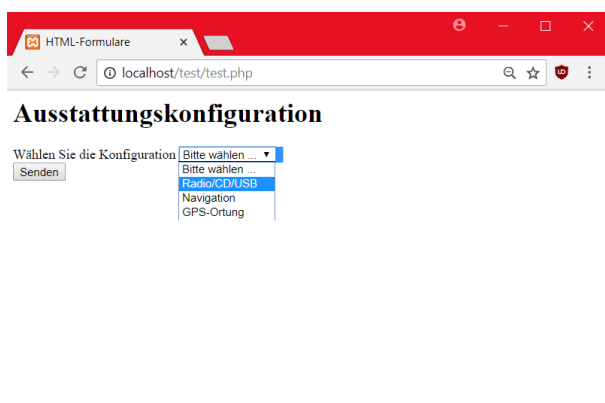


Abbildung 24: Ein Klick auf den Pfeil öffnet ein "Dropdown"-Feld und bietet die jeweils in den <option>-Elementen eingefassten Einträge zur Auswahl an.

Mehrfachselektion?

Mit dem bis hier besprochenen Formular kann maximal ein Element aus der Liste gewählt werden. Das entspräche im Grunde genommen dem gleichen Ergebnis, wie es mit Radio-Button erreichbar wäre. Denkbar ist aber auch eine Mehrfach-Auswahl zuzulassen. Hierzu muss im `<select>`-Element das Attribut „multiple“ eingefügt werden¹⁸.

```
<select name="wahl[]" id="selection" multiple>
```

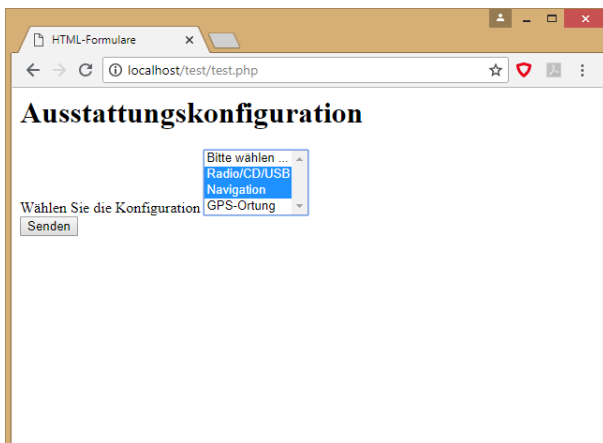


Abbildung 25: Hier ist eine mehrfache Selektion möglich, wenn beim Mausklick zusätzlich die [Strg]-Taste gedrückt wird.

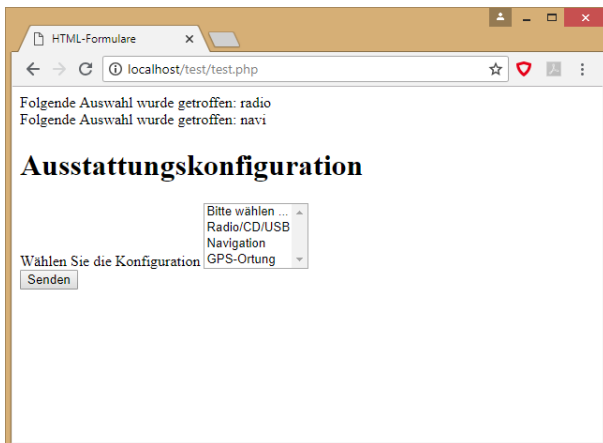


Abbildung 26: Ist im `<select>`-Element das Attribut "multiple" gesetzt, so ist mit zusätzlichen Druck der [Strg]-Taste eine Mehrfachauswahl möglich.

¹⁸ In reinem HTML-Code kann das Attribut ohne Wert direkt in das Element geschrieben werden. In älteren Skripten, die XHTML-konform programmiert wurden, findet man die folgende Schreibweise, die von modernen Browsern ebenso verstanden und korrekt interpretiert wird: `multiple="multiple"`

Verschachtelte Auswahllisten

Die Inhalte der Auswahllisten lassen sich in Gruppierungen zusammen fassen. Die Grundstruktur des HTML-Codes bleibt dabei erhalten:

- Ein `<select>`-Element umschließt als Container die gesamte Auswahlliste.
- Die auswählbaren Elemente werden jeweils in `<option>`-Elementen in die Liste eingefügt.

Das Element `<optgroup>` fügt Zwischenüberschriften in die Liste ein. Dazu wird `<optgroup>` als Container verwendet. Es werden also alle `<option>`-Elemente des gleichen Themas zu einer Gruppe zusammengefasst.

```
<select name="wahl[]" id="selection">
  <optgroup label="HiFi/GPS">
    <option value="radio">Radio/CD/USB</option>
    <option value="navi">Navigation</option>
    <option value="gps">GPS-Ortung</option>
  </optgroup>
  <optgroup label="Tuning">
    <option value="lack">Sonderlackierung</option>
    <option value="lenkrad">Sportlenkrad</option>
    <option value="felgen">Leichtmetallfelgen</option>
  </optgroup>
</select>
```

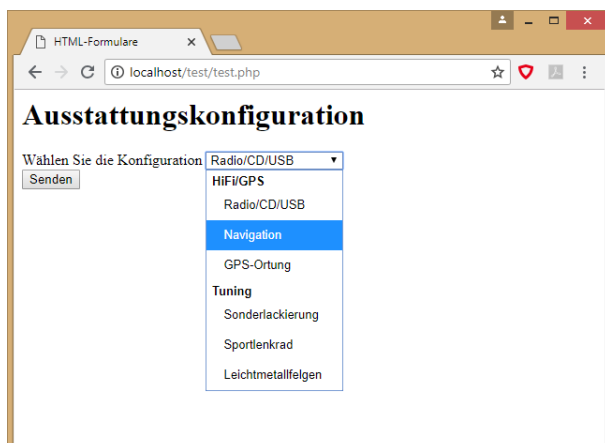


Abbildung 27: Mit dem zusätzlichen Element `<optgroup>` kann eine Auswahlliste detailliert gegliedert werden.

Vorselektion mit dem Attribut „selected“

Sowohl bei Radio-Button und Checkboxes als auch bei Auswahllisten kann mit dem Attribut „selected“¹⁹ eine Vorauswahl angeboten werden. Der Besucher der Seite kann die Vorauswahl bei Bedarf problemlos ändern und eine eigene Wahl treffen. Wird er nicht aktiv, so wird dieser Default-Wert im Formular übernommen und an das weiterverarbeitende Skript geschickt.

Beispiel:

Das eben gesehene Skript wird in einer Zeile verändert:

```
option value="lack" selected>Sonderlackierung</option>
```

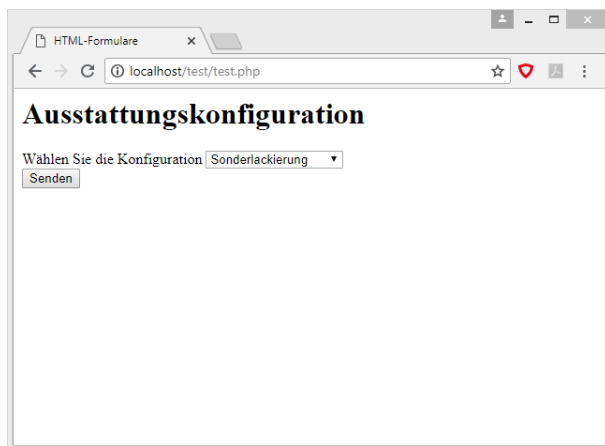


Abbildung 28: Die Vorauswahl mit dem Attribut "selected" in einem der <option>-Elemente wird beim Aufruf des Formulars sofort wirksam, kann aber vor dem Absenden vom Besucher der Seite geändert werden.

Eingabe von Zahlen und Spezialdaten

Verschiedene Daten sind einer klar definierten Schreibweise unterworfen. Das macht es leicht, bereits bei der Eingabe der Daten in ein Formular – ohne Verwendung zusätzlicher Skripte – die Richtigkeit der Form zu überprüfen. So gehört das Zeichen „@“ zu einem klaren Merkmal einer E-Mail-Adresse, dem eine gültige Domain folgen muss. Telefonnummern kennen keine Buchstaben usw. Durch die Wahl eines geeigneten Type-Argumentes in einem <input>-Element kann eine Vorgabe für bestimmte Eingabeformate getroffen werden.

¹⁹ Auch hier ist zu beachten, dass die XHTML-Schreibweise dieses Attributes folgendermaßen aussieht: selected="selected"

type = „number“

Soll ausschließlich die Eingabe reiner Zahlenwerte akzeptiert werden, so bietet sich das `<input>`-Element mit dem Typ „number“ an. Es werden ausschließlich rein numerische Werte akzeptiert. Die Eingabe von Buchstaben und Sonderzeichen ist nicht möglich. Die Reaktionen auf eine Falscheingabe zeigen sich aber abhängig vom verwendeten Browser sehr unterschiedlich: Während beim Chrome schlicht und einfach keine ungültigen Zeichen angenommen werden, reklamiert der Firefox-Browser die geforderte Eingabe mit einem Tooltip und gibt beim Absenden eine entsprechende Meldung aus. Der Internet Explorer verwirft jede falsche Eingabe kommentarlos. Dies kann beim Ausfüllen des Formulars zu Eingabefehlern führen.

```
<input type="number" name="wahl">
```



Abbildung 29: Buchstaben müssen draußen bleiben! - So jedenfalls legt es der Programmierer fest, wenn der Typ "number" für ein Eingabefeld verwendet wird. Es werden nur Zahlen akzeptiert. Mit den Pfeilen kann schrittweise auf- oder abgezählt werden.

Die Möglichkeiten zur Eingabe in einem numerischen Formularfeld lassen sich weiter einschränken. Beispielsweise können obere (max) und untere (min) Grenzen für einen gültigen Wertebereich vorgeben. Zahlen, die außerhalb dieses Bereiches liegen, werden nicht akzeptiert.

Darüber hinaus – dies ist insbesondere für die Eingabe mit der Maus vorteilhaft – können mit dem Attribut „step“ Schrittweiten vorgesehen werden. Im folgenden Beispiel werden Zahlen von 100 bis 1000 akzeptiert, die jedoch in 50er Schritten darstellbar sein müssen.

```
<input type="number" name="wahl" id="test" min="100"
max="1000" step="50">
```

Aufgabe:

Ändern Sie die Grenzwerte auf nicht durch 50 teilbare Werte und experimentieren Sie mit den Eingaben.



Abbildung 30: Es wurden in den Attributen Mindest- und Maximalwerte festgelegt. Hier wurde eine Eingabe versucht, die außerhalb der festgelegten Grenzen liegt.

Die folgenden Abbildungen zeigen wie der Google Chrome-Browser und der MS-Internet Explorer auf eine Falscheingabe reagieren. In beiden Fällen wurde ein Wert eingegeben, der von der vorgesehenen Schrittweite (step="50") abweicht. Darüber hinaus spielt hier der untere Grenzwert eine wichtige Rolle. Von diesem ausgehend, werden die gültigen Schritte abgeleitet. Wird kein Mindestwert vorgegeben, so sind alle Schritte basierend auf den Grundwert Null (in positiver und negativer Richtung) gültig.



Abbildung 31: Es wurde in den Attributen eine Schrittweite (step) von 50 festgelegt. Der Browser akzeptiert deswegen keine Eingabe, die innerhalb der definierten Grenzen nicht auch der Schrittweite entspricht. Der Internet Explorer (rechtes Bild) verwirrt allerdings mit seiner Meldung, ein gültiges „Datum“ einzugeben, auch wenn nicht das Kalenderdatum, sondern der lateinisch korrekte Begriff gemeint ist.

type = „range“

Die Eingabe tendenzieller Werte – bei der Verwendung reiner HTML-Elemente sind hier eher Schätzwerte zu verstehen – kann mit dem Typ „range“ in der Form eines Schiebereglers sehr einfach gestaltet werden.

```
<label for="bereich">Bitte wählen Sie! </label>
<input type="range" min="1" max="5" value="3" id="bereich"
name="wahl"></p>
```



Abbildung 32: Mit dem Schieberegler, der mithilfe des Typs "range" erzeugt wird, lassen sich einfache Werteingaben ermöglichen. Eine Anzeige des aktuellen Wertes gibt es hier allerdings nicht ohne Weiteres.

type = „tel“

Beim Type "tel" handelt es sich im Grunde genommen um ein reines Textfeld. Dies zeigt sich sehr deutlich bei den gängigen Browsern für den PC. Auf verschiedenen mobilen Geräten wird allerdings unterstützend für die Eingabe ein Ziffernfeld angeboten, das einer Wähltastatur ähnlich ist. Dennoch darf nicht von einer Vor-Validierung der Eingabe ausgegangen werden.

Mit dem Attribut „value“ lassen sich allerdings Teile der Rufnummer oder Rufnummernschablonen definieren, die das Feld vorbesetzen. Hier müssen allerdings im weiter verarbeitenden Skript mögliche Eingabefehler abgefangen werden, wenn die Schablone nicht oder nur teilweise überschrieben wird.

```
<input type="tel" value="+43/" name="wahl">
```

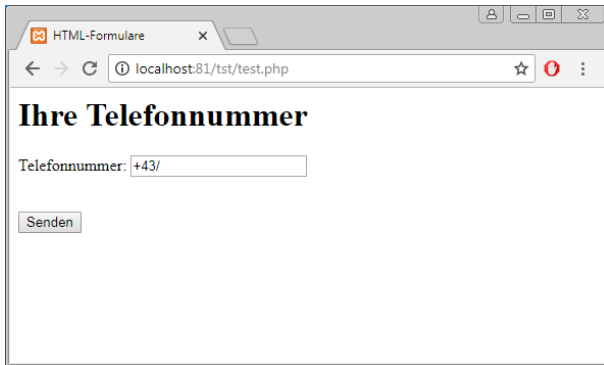


Abbildung 33: Eingabefelder für Telefonnummern erscheinen wie gewöhnliche Texteingabefelder. Um es dem User einfacher zu machen, können mit dem Argument "value" bereits Beispiele oder Rufnummernteile eingesetzt werden.

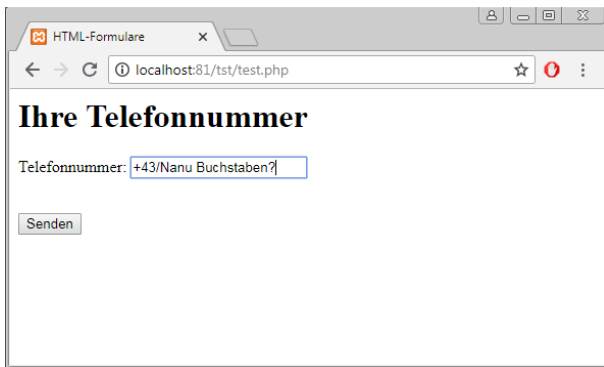


Abbildung 34: Leider ist das Eingabefeld vom Typ "tel" nicht besonders restriktiv. Es können beliebige alphanumerische Zeichen eingegeben werden.

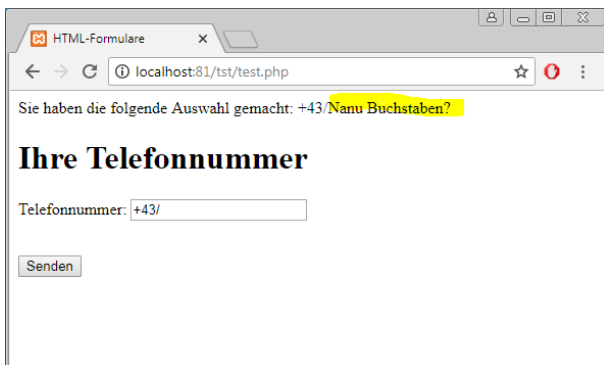


Abbildung 35: Hier wurde ganz sicher keine Rufnummer eingetragen und doch wurde die Eingabe akzeptiert. Dies muss bei der Programmierung von Formularen und der Auswertung der Daten berücksichtigt werden.

type = „email“

Der Typ "email" des <input>-Elementes bringt eine eingeschränkte Validierung der Eingabe mit sich. So wird überprüft, ob ein „@“-Zeichen in der Zeichenkette enthalten ist. Darüber hinaus darf die Zeichenkette nach diesem „@“-Zeichen nicht enden. Dies ist allerdings nicht wirklich ausreichend und so

sind Falscheingaben dennoch möglich. Um eine – aus Sicht der automatischen Validierung – „gültige“ E-Mail-Adresse in das Formular einzufügen, genügt es, lediglich ein einziges Zeichen nach dem „@“ zu schreiben. Ob rechts vom „@“-Zeichen eine – formell(!) – gültige Domain geschrieben wurde, prüfen die Browser nicht.

Ebenso wird die doppelte Verwendung eines „@“-Zeichens innerhalb der Zeichenkette eine Ablehnung der Eingabe bzw. Fehlermeldung bewirken.

```
<input type="email" name="wahl">
```

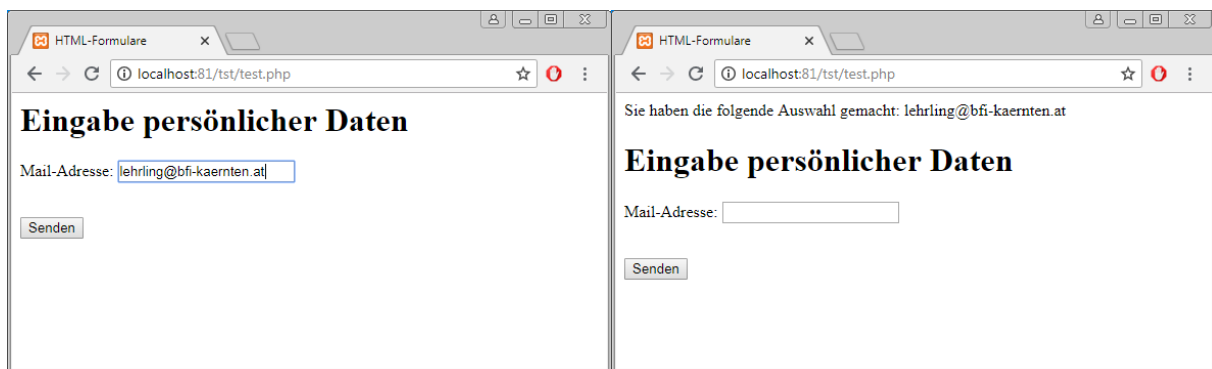


Abbildung 36: Ein Eingabefeld für eine E-Mail-Adresse erscheint zunächst wie ein gewöhnliches Eingabefeld für einen Text. Erfüllt die Eingabe die wenigen Validierungsregeln für eine E-Mail-Adresse, so wird die Eingabe akzeptiert (rechte Seite).

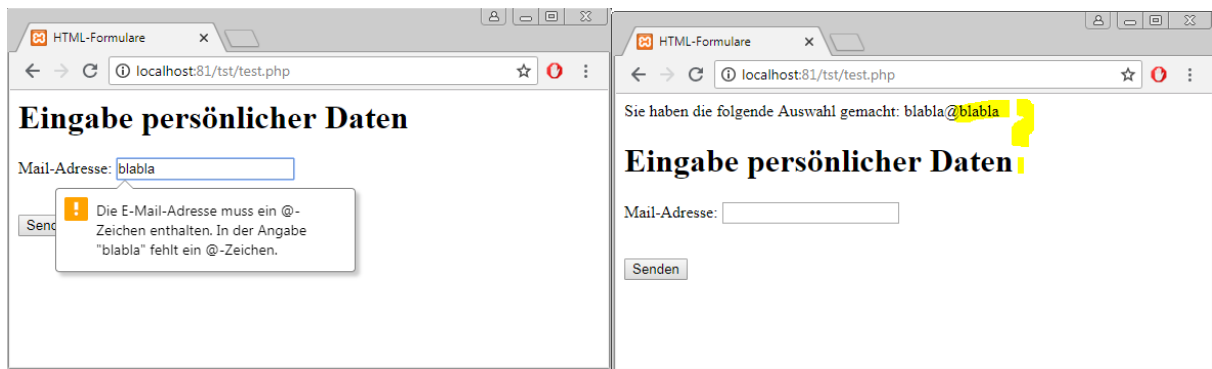


Abbildung 37: Das Eingabefeld vom Typ "email" prüft, ob ein "@"-Zeichen in der Adresse vorhanden und ob danach weiterer Text folgt. Nur dann wird die Eingabe akzeptiert. Doch Vorsicht! - "Nobody is perfect!" - Das gilt auch für die Prüfung einer E-Mail-Adresse bei der Eingabe in ein HTML-Formular. Eine detaillierte Prüfung der Domain erfolgt nicht.

type = „url“

Ebenso wie beim Typ „email“ wird die Eingabe in ein <input>-Feld vom Typ „url“ einer oberflächlichen Validierung unterzogen. Genau genommen wird lediglich überprüft, ob der Protokollpräfix im URL enthalten ist oder nicht. So wird die Webadresse

www.bfi-kaernten.at

vom Browser abgewiesen. Korrekt ist

<http://www.bfi-kaernten.at> oder **<https://www.bfi-kaernten.at>**

eine weitere Überprüfung erfolgt nicht. So ist deswegen aus der Sicht des HTML-Formulars auch <http://murks> ein „gültiger“ URL.

```
input type="url" name="wahl">
```

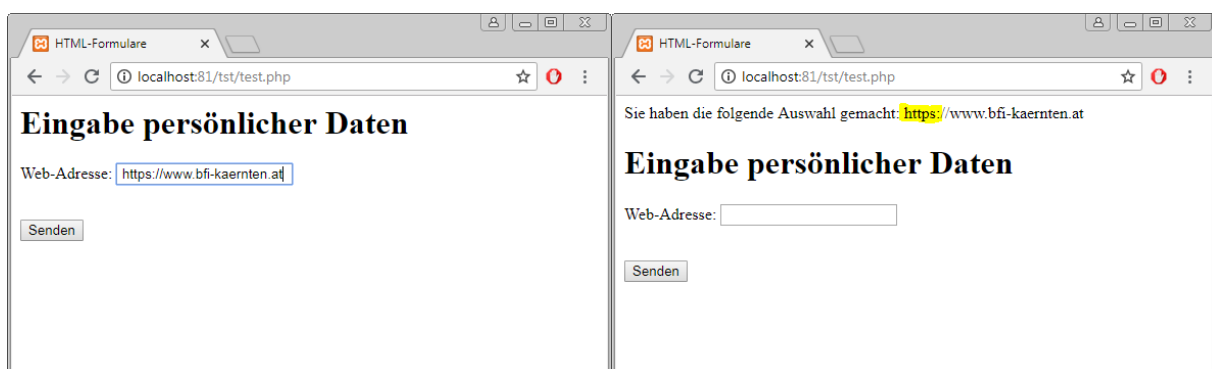


Abbildung 38: Eingabe einer Web-Adresse in ein Eingabefeld vom Typ "url". Es wird eine vollständige Eingabe erwartet! Das heißt: Auch der Protokoll-Präfix im URL ist Bestandteil der Adresse!

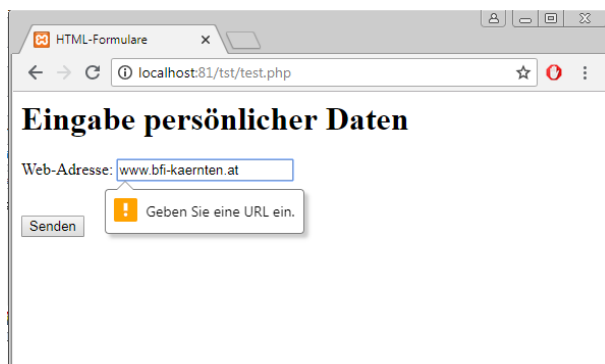


Abbildung 39: Der Typ "url" eines Eingabefeldes fordert eine vollständige Webadresse inkl. dem Protokoll-Präfix (<http://> oder <https://>). Fehlen diese Informationen, wird die Eingabe nicht akzeptiert.



Abbildung 40: Eine besondere Prüfung der formalen Schreibweise eines URLs erfolgt - abgesehen vom Protokoll-Präfix - nicht.

Versteckte Eingabe: type = "hidden"

Ein insbesondere in dynamischen Webformularen sehr wichtiger <input“-Typ ist „hidden“. Mit „type=“hidden“ wird ein *verstecktes Feld* in das Formular eingefügt. Der User kann dieses weder sehen noch mit eigenen Daten ausfüllen.

Der Sinn wird deutlich, wenn ein Webformular bereits mit vorbearbeiteten Daten arbeiten soll, die dynamisch aus einer Datenbank oder vorangegangenen Abfragen übernommen wurden. Der im folgenden Beispiel grün betonte Teil²⁰ (Attribut „value“) wird dann natürlich von einem Skript mit Inhalten belegt.

```
...
<p><input      type="checkbox"      name="wahl[]"      value="GPS"
id="button3">
<label for="button3"> GPS-Ortung</label></p>
<input  type="hidden"  name="wahl[]"  value="versteckte Eingabe:
Autoschlüssel">
...
```

²⁰ Das Skript ist eine Ergänzung des HTML- und PHP-Codes auf Seite 26.

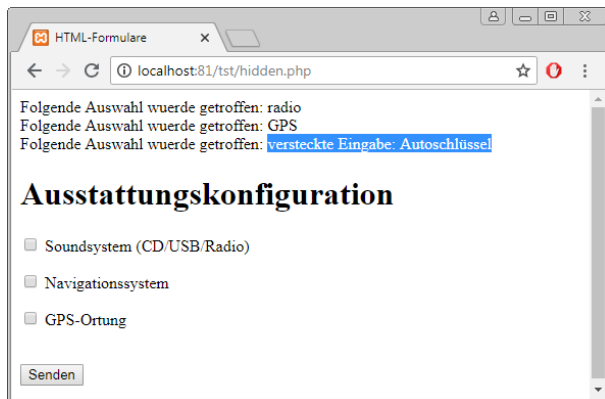


Abbildung 41: Das Ergebnis-Array wurde neben der direkten User-Eingabe über (hier) Checkboxes auch um eine Information aus einem versteckten Eingabefeld ergänzt.

Dateiupload

Bewerberportale ohne die Möglichkeit, einen Lebenslauf auf den Server zu laden sind ebenso undenkbar wie soziale Netzwerke, die von persönlichen Fotos und Dokumenten „leben“. Aus der Sicht der HTML-Seite ist der Datei-Upload sehr einfach und mit einem simplen `<input>`-Element mit dem Typ „file“ realisierbar. Auf der Serverseite gestaltet sich alles ein wenig komplizierter, denn hier geht es nicht zuletzt auch um Schreibrechte im System.

type = "file"

Mit dem Typ „file“ des `<input>`-Elementes wird ein Button in das Formular gesetzt, mit dessen Hilfe ein Auswahldialog für eine Datei auf dem lokalen Rechner geöffnet werden kann. Sobald der Senden-Button betätigt wird, startet der Upload dieser Datei auf den Webserver. Optional kann das Argument „size“ verwendet werden. Dieses legt allerdings nur die Breite eines möglicherweise eingeblendeten Eingabefeldes²¹ fest nicht jedoch die maximale Dateigröße.

```
<input type="file" name="upload[]">
```

²¹ Nicht jeder Browser blendet hier ein Textfeld ein.

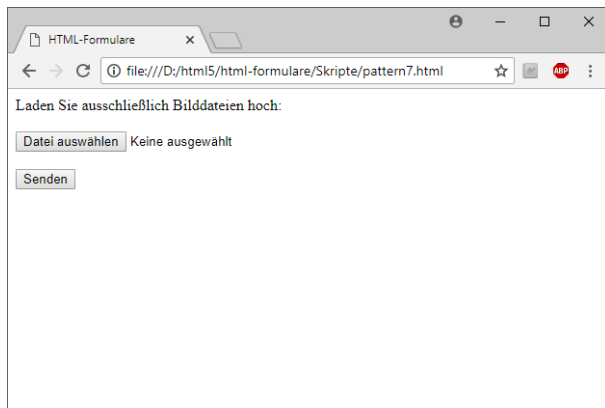


Abbildung 42: Das `<input>`-Element vom Typ "file" blendet einen Button ein, über den ein Explorerfenster zur Auswahl einer Datei erreicht wird. Daneben wird der Name der gewählten Datei bzw. ein Leer-Hinweis ausgegeben.

Einschränkung mit dem Attribut "accept"

Mit dem Attribut "accept" lässt sich eine Einschränkung auf bestimmte Dateiformate vornehmen. In der Regel wird hier mit den so genannten MIME-Bezeichnungen eine Klassifizierung des gewünschten Dateityps vorgenommen. MIME steht für *Multipurpose Internet Mail Extension* (definiert in RFC 6838²² der IETF). Die Bezeichnung besteht aus zwei Teilen: Der erste Teil beschreibt grob die Art der Datei (z. B.: image, text, application etc.). Mit dem durch ein Slash getrennten zweiten Teil wird der konkrete Dateityp (z.B.: jpeg, png, txt, html, pdf etc.) klassifiziert. Ein Asterisk (*) steht als Platzhalter für einen beliebigen Typ.

```
<input type="file" name="upload[]" accept="image/*">
```

²² <https://tools.ietf.org/html/rfc6838>

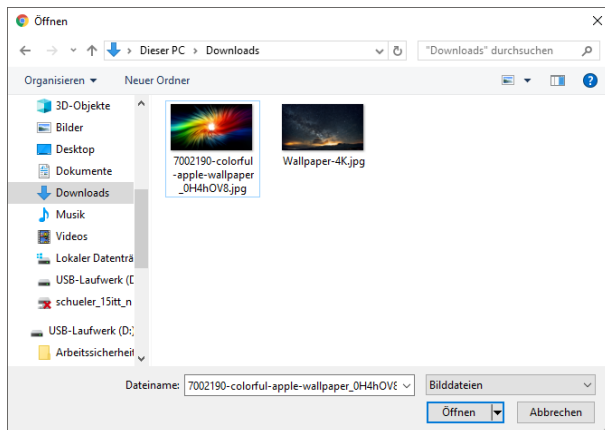


Abbildung 43: Die Eingrenzung des Dateityps mit dem Argument `accept="image/*"` führt dazu, dass der Auswahldialog ausschließlich „Bilddateien“ anzeigt. Ist nicht ausdrücklich der Upload mehrerer Dateien vorgesehen, lässt sich lediglich eine einzige Datei auswählen.

Mehrere Dateien hochladen

Wird das Eingabeelement um das Attribut „multiple“ erweitert, ist auch der Upload mehrerer Dateien gleichzeitig möglich.

```
<input type="file" name="upload[]" multiple>
```

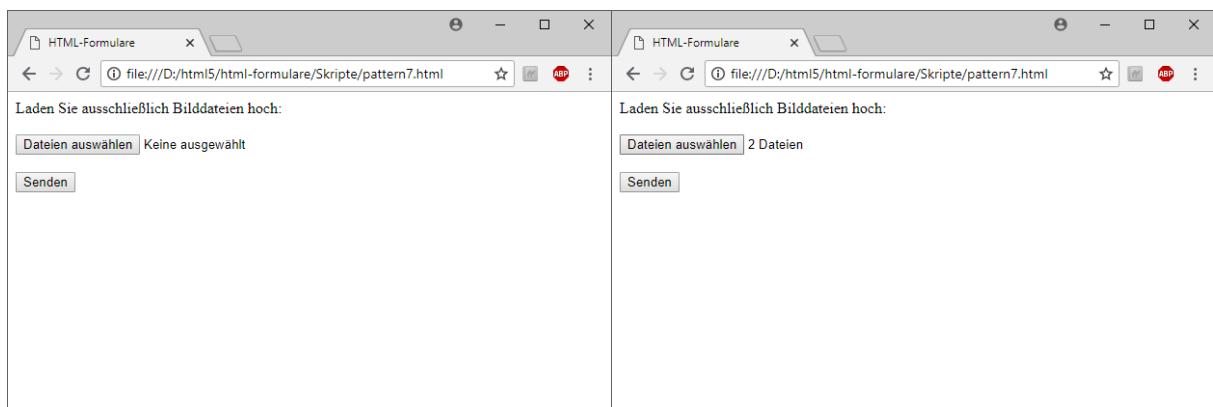


Abbildung 44: Beim Upload mehrerer Dateien wird die Anzahl der gewählten Dateien angezeigt.

Exkurs: Was passiert auf der Serverseite?

Thema dieses Kurses sind HTML-Formulare. Allerdings reicht es meist nicht aus, im Frontend eine Upload-Möglichkeit für Dateien in ein Formular einzubauen. Es ist natürlich auch wichtig, diese „Eingabe“ richtig verarbeiten zu können. Das

ist etwas komplexer und setzt einen entsprechend konfigurierten Server²³ voraus. An dieser Stelle soll das grundlegende Prinzip kurz erläutert werden.

Wenn das Formular nach der Auswahl der zu übertragenden Datei (bzw. den zu übertragenden Dateien) abgeschickt und auf der Serverseite empfangen wird, wird dort ein Array angelegt: `$_FILES['formulardateiname']`²⁴. In dieses werden die folgenden Informationen gespeichert.

- `$_FILES['formulardateiname']['name']`: Hier findet sich der ursprüngliche Dateiname wieder.
- `$_FILES['formulardateiname']['type']`: MIME-Typisierung des Dateityps (z.B.: image/png, text/plain oder text/html etc.)
- `$_FILES['formulardateiname']['tmp_name']`: Name der Datei, wie sie temporär auf dem Webserver gespeichert wird. Diese Datei befindet sich noch nicht im vorgesehenen Zielverzeichnis! Sie muss von der verarbeitenden Applikation dorthin kopiert werden.
- `$_FILES['formulardateiname']['error']`: Indikator, ob während des Upload-Vorganges ein Fehler festgestellt wurde.
- `$_FILES['formulardateiname']['size']`: Dateigröße in Byte

Mit diesen Informationen kann nun ein Skript bzw. eine Applikation auf dem Webserver – die erforderlichen Rechte vorausgesetzt – die Datei aus dem temporären Verzeichnis in das vorgesehene Zielverzeichnis speichern. Dabei muss während des Kopiervorganges auch der ursprüngliche Dateiname, der ebenfalls im Array `$_FILES` gespeichert wird, wiederhergestellt werden.

Zum Kopieren einer Datei mit PHP kann beispielsweise die Funktion `copy()` verwendet werden (in der Praxis bitte auf die Pfadangaben achten):

```
<?php
$quelldatei = 'temp0001.tmp';
```

²³ Wichtig für einen erfolgreichen Datei-Upload sind die richtigen Schreibrechte des Webserver auf dem System: Der Webserver (z. B.: Apache oder IIS) muss in der Lage sein, eine empfangene und temporär gespeicherte Datei zu lesen und an den vorgesehenen Ort im System mit dem richtigen Namen zu kopieren. Das bedeutet auch, dass beispielsweise ein PHP-Skript das Recht haben muss, einen Kopierbefehl auf Systemebene auszuführen.

²⁴ Der hier verwendete Begriff 'formulardateiname' entspricht dem Wert des Attributes „name“ im <input>-Feld vom Typ „file“. Im Beispiel zur Erläuterung des Typs „file“ wurde als Name „upload“ verwendet.

```
$zieldatei = 'produktfoto.png';

if (!copy($quelldatei, $zieldatei)) {
    echo 'Kopieren der Datei '.$zieldatei.' Ist gescheitert!';
}

?>
```

Die Funktion `copy()` gibt einen logischen Wert zurück (0 oder 1 bzw. wahr oder falsch), der vom Erfolg des Kopiervorganges abhängt. Das vorangehende PHP-Skript kopiert eine Datei `temp0001.tmp` in eine neue Datei (hier: gleiches Verzeichnis) mit dem Namen `produktfoto.png`. Dies passiert gleichzeitig mit der Auswertung des Ergebnisses durch die `if`-Verzweigung.

Alternativ können auf einem Linux-System auch direkte Shell-Kommandos mit der Funktion `shell_exec()` ausgeführt werden.

Weiterführende Infos:

<https://www.php-kurs.com/upload-dateien-mit-php.htm>

<http://php.net/manual/de/function.copy.php>

<http://php.net/manual/de/function.shell-exec.php>

Auswahl einer Farbe mit `type = „color“`

Der Input-Type „color“ blendet zunächst ein kleines und sehr unscheinbares Farbfeld in das Formular ein. Klickt man darauf, öffnet sich jedoch ein sehr vielseitiger Farbeditor. Neben den Grundfarben können beliebige Farben aus dem gesamten Spektrum gewählt oder durch Vorgabe der RGB-Werte²⁵ individuell definiert werden. Individuelle Farben lassen sich in einer Palette „Benutzerdefinierte Farben“ speichern. Neben der reinen Farbwahl ist es möglich, auf den Farbton, die Sättigung und die Helligkeit direkten Einfluss zu nehmen.

```
<input type="color" id="test" name="farbe">
```

²⁵ RGB steht für Rot, Grün und Blau. Für diese Grundfarben des Farbspektrums, aus denen sich alle weiteren Farben mischen lassen, können Farbwerte zwischen 0 und 255 vergeben werden.

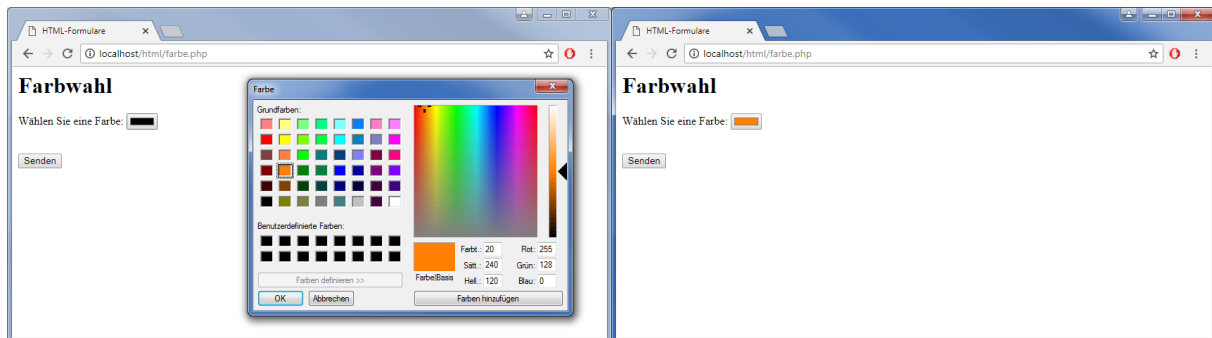


Abbildung 45: Mit dem Input-Type "color" kann der hexadezimale Code jeder in HTML definierten Farbe mit wenigen Mausklicks gewählt werden.

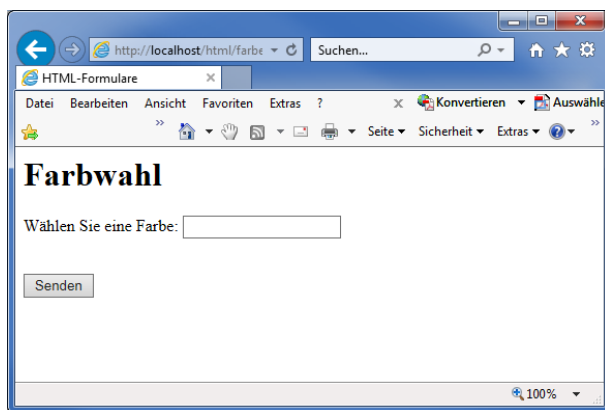


Abbildung 46: Leider funktionieren nicht alle HTML-Elemente mit dem Internet Explorer (hier: IE11).

Ein kleines PHP-Skript (im folgenden Listing sind alle PHP-Code-Zeilen blau und alle reinen HTML-Zeilen orange geschrieben) soll zeigen, wie die Farbwahl direkt in einer Webseite eingesetzt werden kann.

Der erste PHP-Codeblock im Listing ist im Prinzip schon aus den vorangegangenen Beispielen bekannt: Hier wird abgefragt, ob `$_POST['farbe']` bereits mit einem Wert belegt ist, was beim erstmaligen Aufruf der Seite natürlich noch nicht der Fall sein kann.

Interessant ist die kleine PHP-Zeile innerhalb des HTML-Bereiches: Diese bewirkt lediglich die Ausgabe des Inhaltes von `$_POST['farbe']`. Ist `$_POST['farbe']` leer, ist das `Style_Attribut` ungültig und bewirkt keine Änderung im HTML-Code. Eine Vorabprüfung auf einem vorhandenen Wert ist deswegen nicht nötig. Enthält `$_POST['farbe']` jedoch einen zuvor im Formular gewählten Farbcode, dann wird dieser Wert in der CSS-Eigenschaft „color“ wirksam und verändert die Farbe der Überschrift.

`<?php`

```

if ($_POST['farbe'])
echo 'Sie haben die folgende Farbe gewählt: ' . $_POST['farbe'];
?>

<form method="POST">
<h1 style = "color: <?php echo $_POST['farbe'] ?>">
Farbwahl</h1>
  <p>
    <label for="test">Wählen Sie eine Farbe: </label>
    <input type="color" id="test" name="farbe">
  </p>
  <br>
  <button type="submit">Senden</button>
</form>

```

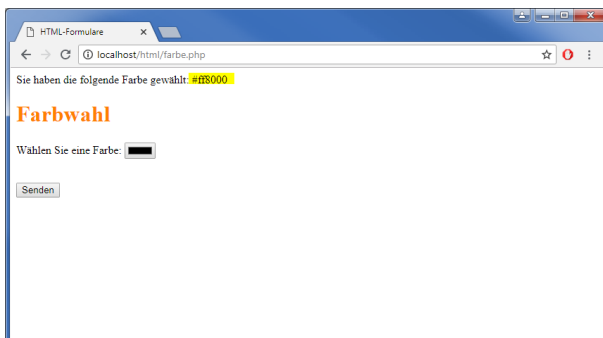


Abbildung 47: Im `$_POST`-Array wird der hexadezimale Farbcode gespeichert. Dieser wird hier im Beispiel mit einem kleinen PHP-Skript direkt in einem `Style`-Attribut zur CSS-Formatierung der Überschrift verwendet.

Schaltflächen

Schaltflächen sind für ein HTML-Formular sehr wichtig und mindestens eine Schaltfläche ist grundsätzlich Bestandteil des Formulars. In den bisher gezeigten Beispielen war stets ein Button vorhanden, mit dessen Hilfe die Formulardaten an das weiter verarbeitende Skript abgeschickt wurden. Schaltflächen können sowohl mit dem Element `<input>` als auch mit dem Element `<button>` erzeugt werden. Letzteres ist der aktuelle Stand der Definition, wobei beide Verfahren offiziell zulässig sind.

Das Element <button>

Mit dem Element <button> werden Schaltflächen für ein Formular erzeugt. Die beiden elementaren Schaltflächen dienen dem Versand der Formulareingaben und dem Zurücksetzen der Eingaben. Ein drittes Element ist für die individuelle Gestaltung vorgesehen. Die Kennzeichnung bzw. Beschreibung der Button kann als Text ebenso wie mit grafischen Elementen erfolgen.

type = „submit“

Das Element <submit> erzeugt standardmäßig einen „Submit“-Button (Versenden des Formular-Inhaltes), wenn ihm kein type-Attribut zugewiesen wird. Im Gegensatz zum <input>-Element des gleichen Typs muss die Beschriftung des Button als Element-Inhalt zwischen dem öffnenden und dem schließenden Tag eingetragen werden. Damit ist ohne Verwendung weiterer Attribute eine sehr flexible Gestaltung des „Submit“-Button möglich. Beispiele sind u.a.:

```
<button type="submit">Senden</button>
<button type="submit">Abschicken</button>
<button type="submit">Submit</button>
<button type="submit">Jetzt Kaufen</button>26
<button type="submit">... und tschüss!</button>
```

Ebenso kann anstelle eines Schriftzuges ein -Element eingefügt werden. Wie bei jeder Illustration einer Webseite sollten die wichtigsten Attribute nicht fehlen: Eine Breiten- oder Höhendefinition sorgt dafür, dass die Schaltfläche nicht zu groß und nicht zu klein in der Seite erscheint. kann das Bild nicht geladen werden, ist das „alt“-Attribut entscheidend, um dem Besucher der Seite über den Alternativtext die Funktion der Schaltfläche zu erläutern. Das Attribut „title“ gehört aus Gründen der heute wie selbstverständlich erwarteten Barrierefreiheit zum „guten Ton“ in Programmiererkreisen. Es erzeugt ein „Tooltip“, das beim Überfahren mit dem Mauszeiger eine Erklärung einblendet.

²⁶ Bei einem Online-Shop müssen Schaltflächen, deren Aktivierung einen Geldfluss bzw. eine Vermögensbewegung und vertragliche Verpflichtungen bewirken eindeutig beschriftet werden. So ist der Schriftzug „Weiter“ nicht ausreichend, wenn ein Kauf ausgelöst wird. Allein dieser „Formfehler“ kann dazu führen, dass ein Online-Geschäft ungültig ist.

```
<button type="submit">
```

```

```

```
</button>
```

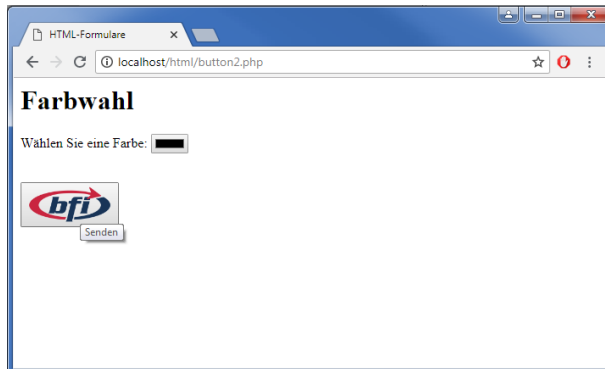


Abbildung 48: Die Schaltfläche zum Versenden des Formularinhaltes (submit) wird hier durch eine individuelle Grafik dargestellt.

type = „reset“

Mit einem <button> vom Typ „reset“ werden alle aktuellen Formularinhalte gelöscht bzw. die Formularinhalte auf den Urzustand zurückgesetzt. Letzteres bedeutet: Ist ein Formularwert bereits mit einem „value“- oder „checked“-Attribut vorgegeben, so wird dieser Zustand auch nach einer Veränderung des Inhaltes oder des Zustandes wieder hergestellt.

```
<button type="reset">Alles Löschen</button>
```

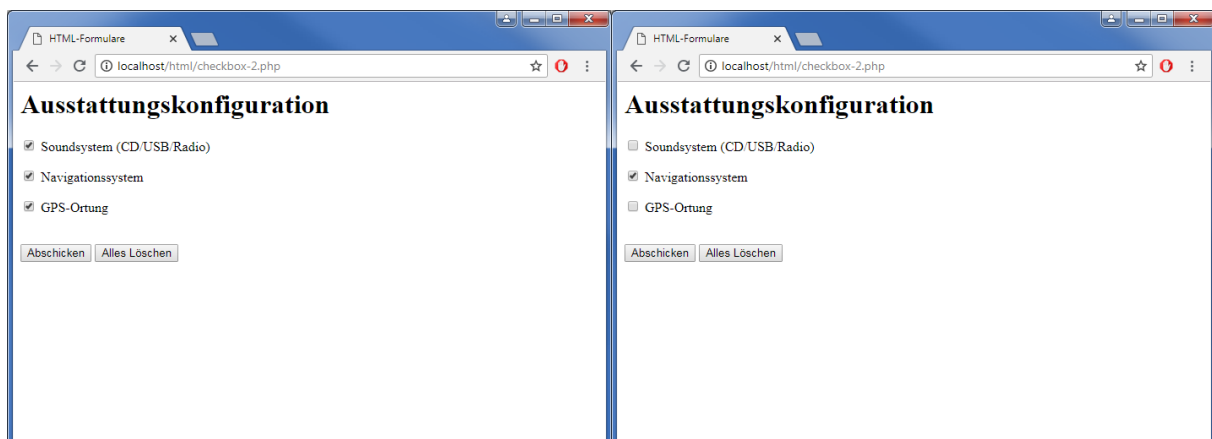


Abbildung 49: Im linken Bild hat der Besucher der Seite irrtümlich alle Felder aktiviert. Anstelle das Formular abzuschicken, klickt er auf die Schaltfläche "Alles Löschen" (Element <button> mit dem Attribut "reset"). Rechts sind nur zwei der drei Checkboxes deaktiviert. Der Grund: Die Checkbox zum „Navigationssystem“ ist mit dem Attribut „checked“ als aktiv vorgegeben.

type = „button“

Nanu, ein Typ „button“ für ein Element <button>? – Ein Button dieses Typs ist keiner Standard-Aktivität zugewiesen, sondern wird für beliebig programmierbare Aktionen eingesetzt. Wichtig: Mit einer Schaltfläche vom Typ „button“ erfolgt noch kein Versand der Formularinhalte! Ein kleines Beispiel soll dies demonstrieren:

Der HTML-Rumpf enthält dieses – sehr einfache – „Formular“ mit drei Schaltflächen:

```
<form name="formular">
    <button type="button" name="hg" value="hgred"
    onClick="hgrot()">roter Hintergrund</button>
    <button type="button" name="hg" value="hggreen"
    onClick="hggr()">grüner Hintergrund</button>
    <button type="button" name="hg" value="hgblue"
    onClick="hgblau()">blauer Hintergrund</button>
</form>
```

Darüber hinaus wird an dieser Stelle zum Umschalten der Hintergrundfarben JavaScript verwendet:

```
<script>
function hgrot() {document.bgColor = "#FF0000";}
function hggr() {document.bgColor = "#00FF00";}
function hgblau() {document.bgColor = "#0000FF";}
</script>
```

Sowohl das <script>- als auch das <style>-Element sind Teil des HTML-Heads.

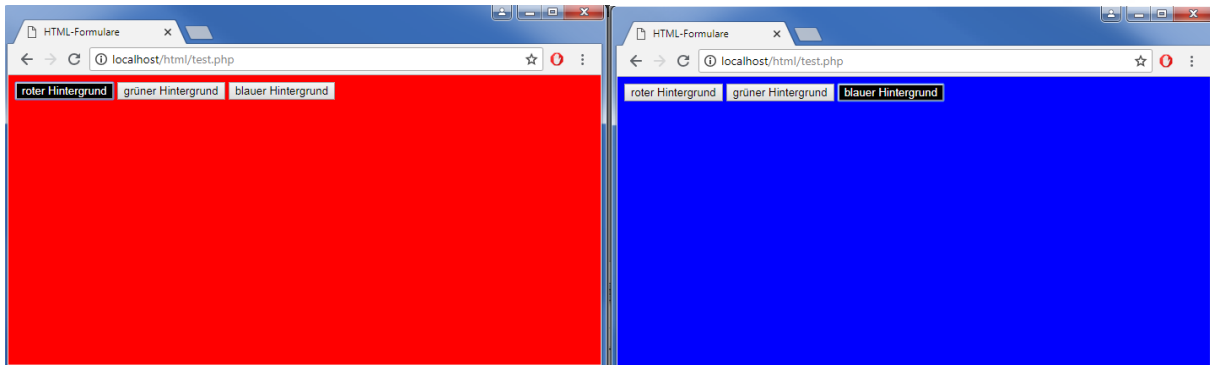


Abbildung 50: Einsatz frei programmierbarer Schaltflächen in einem HTML-Formular: Hier erfolgt lediglich die Umschaltung der Hintergrundfarbe mithilfe kleiner JavaScripte.

Das Formular schaltet nicht nur die aktuelle Hintergrundfarbe um, sondern legt mit den Wertepaaren aus den Attributen „name“ und „value“ auch Daten für ein weiter verarbeitendes Skript fest. Dies wäre natürlich auch bereits mit einem „Submit“-Button möglich gewesen. Allerdings ruft der Submit-Button sofort das Folgeskript auf, was beim benutzerdefinierten Button nicht der Fall ist.

Man kann also mit einem <button>-Element mit dem Typ „button“ sehr flexibel arbeiten und sowohl benutzerdefinierte Aktionen auslösen, ohne das Formular an sich sofort zu versenden als auch Einstellungen vornehmen, wie sie sonst nur mithilfe von Radio-Button oder Checkboxes umsetzbar wären. Allerdings erfordert all dies stets zusätzliche Programmierarbeit.

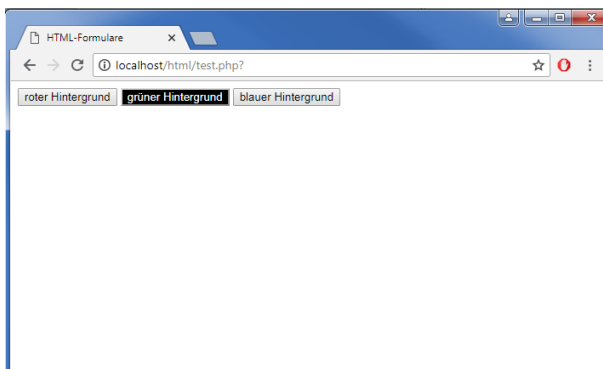


Abbildung 51: Beinahe ein identisches Skript wie im Beispiel zuvor, jedoch wurde hier der Button-Typ "submit" gewählt. Beim Klick auf eine Schaltfläche wechselt der Hintergrund nur für den Bruchteil einer Sekunde die Farbe. Da jedoch die Seite neu aufgebaut wird, bleibt der Hintergrund letztlich weiß.

CSS-Selektor „button“

Wie jedes andere HTML-Element können auch Schaltflächen direkt mit CSS formatiert werden. Der Selektor referenziert auf das Element `<button>`. Das vorherige Beispiel zeigt dies: Um die Schaltflächen optisch für die Screenshots zur Illustration etwas zu betonen, wurde ein CSS-Format verwendet. Dieses wurde im Kopf der Seite in einem `<style>`-Element eingefügt:

```
<style>
button:hover {
    background-color: black
    color: white;
}
</style>
```

Sobald die Maus über eine (hier: beliebige) Schaltfläche geführt wird, bekommt diese eine weiße Schrift auf schwarzer Hintergrundfarbe.

Veraltet: Schaltflächen mit dem `<input>`-Element

Nicht falsch, nicht verboten, aber dennoch nicht mehr die optimale Wahl ist die Verwendung des `<input>`-Elementes für die Erzeugung von Schaltflächen. Der Vollständigkeit wegen sollen die `<input>`-Typen für Schaltflächen an dieser Stelle nicht verschwiegen werden. Empfohlen wird jedoch die Verwendung des `<button>`-Elementes, das auch direkt per CSS formatierbar ist.

Die Syntax unterscheidet sich ebenfalls vom `<button>`-Element. Eine individuelle Beschriftung einer Schaltfläche mit dem `<input>`-Element ist zwar möglich, wird aber mit dem Attribut „value“ realisiert. Im Gegensatz zum `<button>`-Element kann ein mit dem Formular verschickter Wert also beim `<input>`-Element nicht von der Beschriftung unterschiedlich sein.

```
<input type="submit">
```

entspricht

```
<button type="submit">Senden</button>
```

Eine Sonderform stellt die grafische Schaltfläche mit dem `<input>`-Element dar. Hier ist der Typ „image“ definiert. Einen solchen Typ gibt es für das Element `<button>` nicht, denn die Beschriftung wird als Element-Inhalt zwischen Star-

und Schlusstag platziert. So wird beim Element `<button>` ein ganz gewöhnliches `<img>`-Element als Inhalt gesetzt.

Die Schreibweise eines `<input>`-Elements für eine grafische Schaltfläche ähnelt der eines `<img>`-Elementes:

```
<input type="image" src="bfi.png" alt="bfi-Schaltfläche">
```

Mehrzeilige Textfelder: Element `<textarea>`

Die bisher gezeigten Eingabemöglichkeiten waren auf einzeilige Felder beschränkt. Für längere Textinhalte – zum Beispiel für Kommentare zu einem Beitrag – sind mehrzeilige Felder komfortabler. Sie ermöglichen eine bessere Übersicht über den bereits geschriebenen Text und tragen damit zur fehlerfreien und verständlichen Verfassung von Texten in einem HTML-Formular bei.

Für mehrzeilige Texteingaben wird ein neues HTML-Element verwendet: `<textarea>` (= Textbereich).

Wichtiger Hinweis:

Das HTML-Element `<input>` kann als so genanntes „leeres“ Element ohne einen schließenden Tag verwendet werden. Dies ist beim Element `<textarea>` nicht der Fall. Es muss in jedem Fall die Form `<textarea>...</textarea>` gewahrt werden!

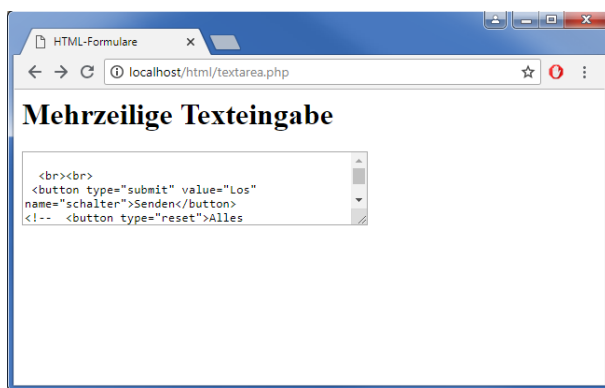


Abbildung 52: Ein `<textarea>`-Element braucht einen schließenden Tag! Sonst werden alle nachfolgenden Code-Zeilen als Vorbelegung betrachtet.

Die Größe eines mehrzeiligen Textfeldes muss unter zwei verschiedenen Gesichtspunkten betrachtet werden: Zum Einen sind die Abmessungen des Eingabefensters zu betrachten, die in der Anzahl von Zeilen (rows) und Spalten

(cols²⁷) definiert wird. Zusätzlich kann eine maximal akzeptierte Textmenge mit dem Attribut „maxlength“ vorgegeben werden. Dies ist sinnvoll, wenn beispielsweise die Zeichenlänge beim Speichern in einer Datenbank begrenzt werden soll. Der das Formular ausfüllende Besucher der Seite erhält bei Überschreitungen sofort ein Feedback.

Während die maximal aufnehmbare Zeichenmenge nicht „verhandelbar“ ist, kann die Größe des Eingabefensters durchaus verändert werden²⁸. Sowohl beim Mozilla Firefox als auch beim Google Chrome-Browser wird am rechten unteren Rand des Eingabebereiches ein „Griff“ eingeblendet, mit dessen Hilfe per Maus das Fenster in der Größe verändert werden kann.

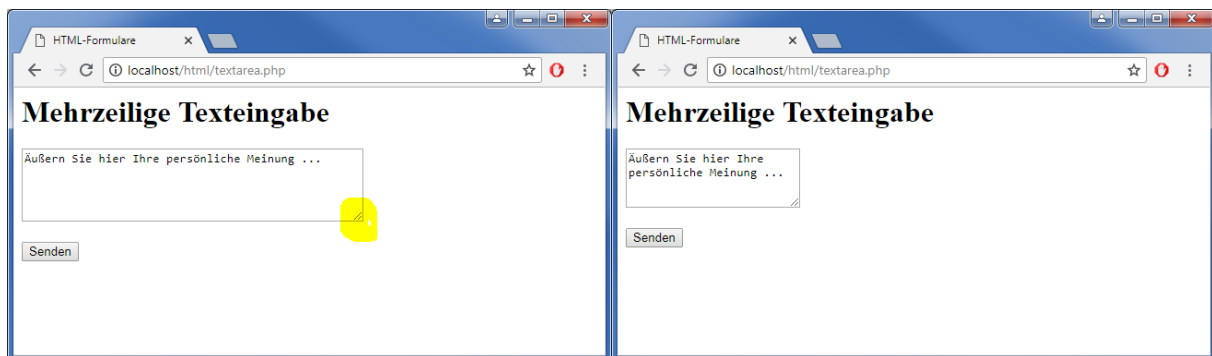


Abbildung 53: Der kleine "Griff" im rechten unteren Eck des Eingabefensters erlaubt dem Besucher die individuelle Veränderung der Größe des Fensters. Auf eine möglicherweise bestehende Begrenzung der eingebaren Zeichenmenge hat dies jedoch keinen Einfluss.

Überschreitet die eingegebene Textmenge – es sei unterstellt, dass die durch das Attribut „maxlength“ festgelegte Grenze noch nicht erreicht ist – dann wird eine kleine Bildlaufleiste²⁹ am rechten Rand des Eingabefeldes eingeblendet. Mit deren Hilfe kann der gesamte Text geprüft und bearbeitet werden, ohne die Größe des Eingabefensters zu verändern.

²⁷ Abgeleitet vom englischen Begriff Column für Spalte.

²⁸ Dies gilt nicht zwingend auch für den MS-Internet-Explorer.

²⁹ Beim Internet Explorer werden Schaltflächen zum Auf- und Abwärtsscrollen aktiviert.



Abbildung 54: Wird mehr Text eingegeben, als durch die Attribute "cols" und "rows" an Platz verfügbar, dann wird das Eingabefenster um eine Bildlaufleiste erweitert. So kann jederzeit auch der möglicherweise nicht sichtbare Text geprüft werden.



Abbildung 55: Die Beschränkungen mit den Attributen "cols" und "rows" beziehen sich lediglich auf die Größe des Eingabefensters. Sie begrenzen nicht die Textmenge, die eingegeben werden kann.

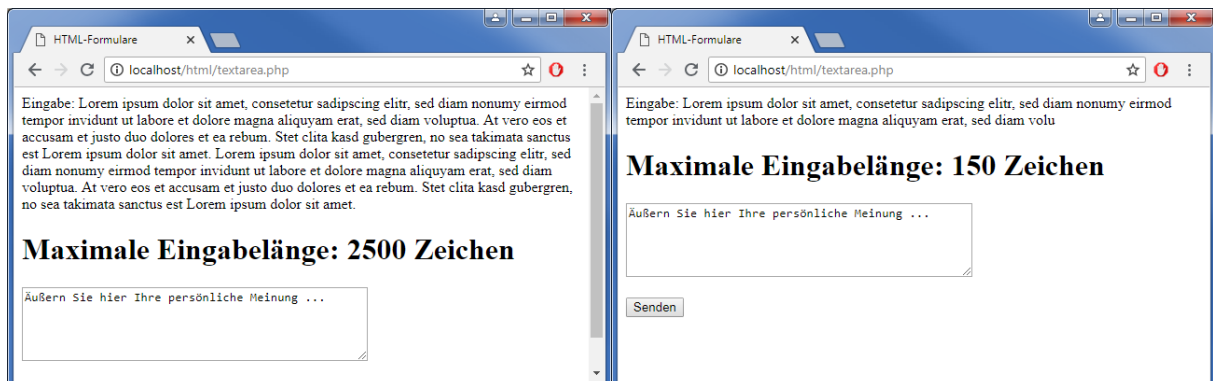


Abbildung 56: Unabhängig von der Größe des Eingabefeldes, die mit den Attributen "cols" und "rows" festgelegt wird, begrenzt das Attribut "maxlength" die Menge des eingebaren Textes bereits auf Formularebene.

Neben dem Attribut „maxlength“, das die maximal eingebbare Zeichenmenge begrenzt, gibt es mit „minlength“ ein Attribut, das eine gewisse Mindestlänge von Text bei der Eingabe erfordert. Damit soll die Eingabe sinnvollen Textes „erzwungen“ und das Ausfüllen des Feldes mit „Dummy-Zeichen“ oder

Emoticons vermieden werden. Einige Browser unterstützen dieses Attribut, jedoch ist dies nicht grundsätzlich voraus zu setzen.

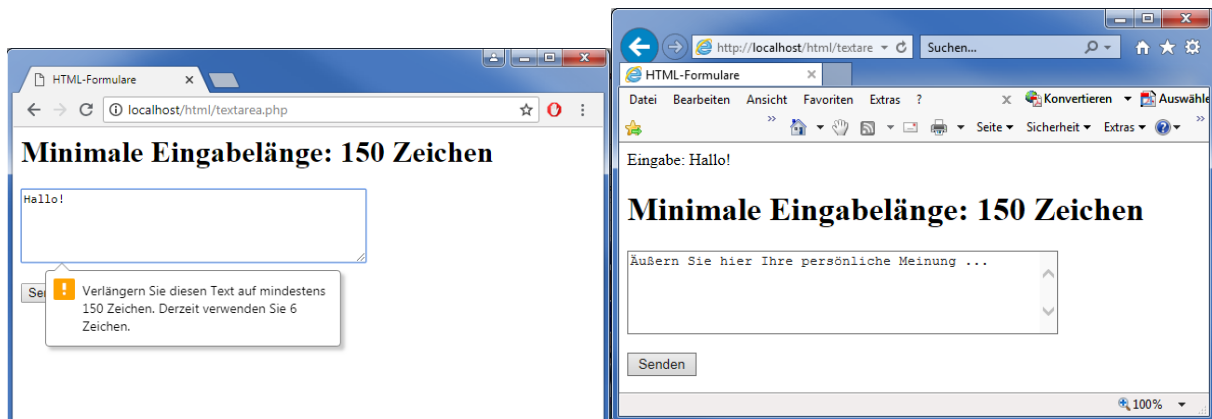


Abbildung 57: Der Chrome-Browser (links) und auch der Firefox-Browser berücksichtigen das Attribut "minlength", mit dessen Hilfe eine Mindesttexteingabe erzwungen werden kann. Dagegen erweist sich der Internet Explorer (hier: IE 11, rechts) als völlig "schmerzfrei" und ignoriert die Anforderung.

Gruppierungen / "Fieldsets"

Das Element <fieldset>

Formulare sind unformatiert eine meist unübersichtliche Ansammlung von Eingabefeldern. Das Element <fieldset> kann helfen, hier für etwas Ordnung zu sorgen.

<fieldset> ist ein Containerelement, in das die Formularelemente – egal, ob es sich um <input>-, <button>- oder <textarea>-Elemente handelt – zu einer Gruppe zusammen gefasst werden, die in einem Kontext zueinander stehen. Das Beispiel (vgl. Abbildung 58) zeigt zwei Formularbereiche wie sie zur Konfiguration einer Autobestellung verwendet werden könnten. Oben im Bild ist die Auswahl der Innenausstattung zu wählen. Das „Fieldset“ besteht aus mehreren Checkboxes (eine Mischung unterschiedlicher Feldtypen ist natürlich möglich). Im Unteren Bereich wurde eine Gruppe von Radio-Button zur Wahl des Fahrzeugtyps in einem „Fieldset“ zusammengefasst.

<fieldset>

<legend>Innenausstattung</legend>

<label for="cd">

```
<input type="checkbox" name="ausstattung[]" id="cd" value="CD-Radio"> CD-Radio
```

```

</label>
<label for="navi">
    <input type="checkbox" name="ausstattung[]" id="navi"
    value="Navigationsgerät"> Navigationsgerät
</label>
<label for="klima">
    <input type="checkbox" name="ausstattung[]" id="klima"
    value="Klimaanlage"> Klimaanlage
</label>
</fieldset>

```



Abbildung 58: Zwei Bereiche des gleichen Formulars sind optisch durch `<fieldset>`-Elemente voneinander getrennt.

„legend“ vs. „label“

Diese Formular-Feldbereiche können eine individuelle Beschriftung erhalten. Dazu ist ein weiteres HTML-Element `<legend>` vorgesehen. `<legend>` existiert neben dem Beschriftungselement `<label>`, wobei letzteres für die einzelnen Felder und `<legend>` für den kompletten Feldbereich gilt.

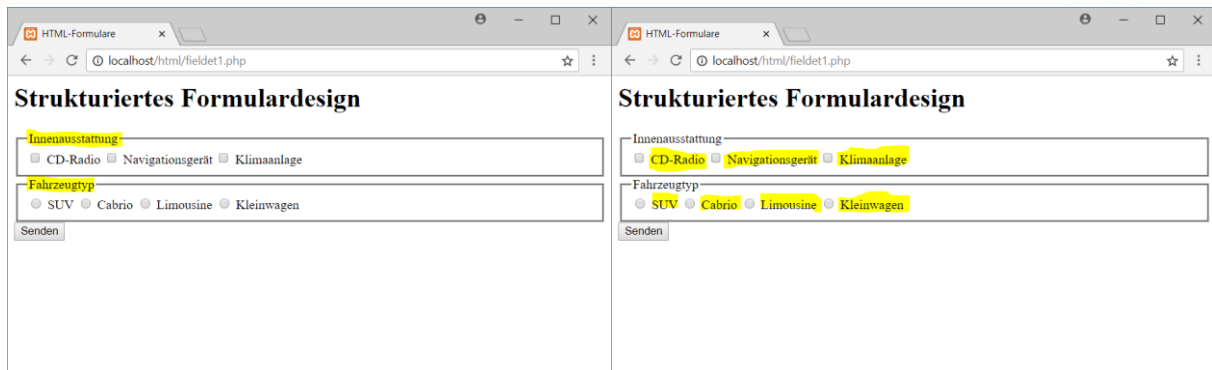


Abbildung 59: Mithilfe des <legend>-Elements können die Fieldsets beschriftet werden (linkes Bild). Mithilfe von CSS lassen sie sich individuell formatieren. Nach wie vor werden die einzelnen Formular-Elemente mit dem <label>-Element beschriftet (rechtes Bild).

Begrenzte Eingaben mithilfe von „Pattern“

Mithilfe von Eingabeschablonen kann in HTML ohne Einsatz eines Skriptes die Eingabe in Formularfelder validiert³⁰ werden. Dazu ist das Attribut „pattern“ vorgesehen. Dessen Verwendung bedarf einiger Aufmerksamkeit, denn der Wert des Attributes kann sehr komplex werden.

Einfaches Beispiel: Rein numerische Eingabe mit 5 Stellen

In das Thema soll mit einem ersten, sehr einfachen Beispiel eingestiegen werden: Die Eingabe einer (deutschen und damit 5stelligen) Postleitzahl. Die hier gezeigten Elemente <label> und <input> werden in ihrem Gebrauch als bekannt voraus gesetzt. Interessant ist das rot betonte Attribut des <input>-Elementes. Es besteht hier aus einem Ausdruck in einer eckigen Klammer und daneben einem Ausdruck in einer geschweiften Klammer.

Der Ausdruck in der eckigen Klammer gibt mit dem Wert „0-9“ vor, dass ausschließlich Ziffern verwendet werden dürfen. Damit wird jede Eingabe abgewiesen, in der versucht wird, einen Buchstaben oder ein anderes Sonderzeichen in das Feld einzugeben.

Im zweiten Ausdruck (in geschweiften Klammern) steht hier lediglich der Wert „5“. Dieser steht für die Zahl der geforderten Stellen. Jede Eingabe, die mehr oder weniger als fünf Stellen umfasst, wird mit dieser Vorgabe zurückgewiesen.

³⁰ Validierung bedeutet „Überprüfung auf Gültigkeit“. – Dies ist im Rahmen gewisser Grenzen möglich und hängt wesentlich von der Qualität der Patterndefinition ab.

```
<label for="plz">Postleitzahl (5stellig!): </label>
<input type="text" name="plz" id="plz" pattern="[0-9]{5}"
placeholder="12345">
```

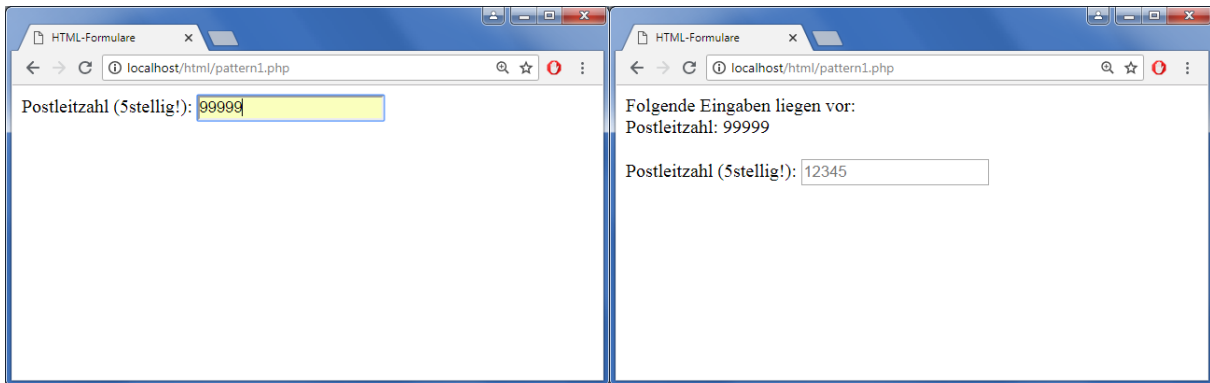


Abbildung 60: Das Bild rechts bestätigt, dass die Eingabe im linken Bild tatsächlich der Vorgabe entspricht.

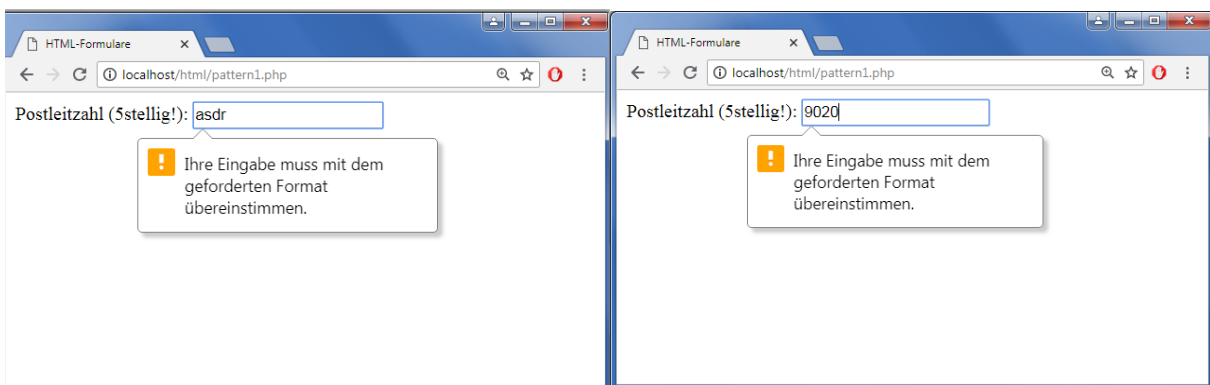


Abbildung 61: Im Bild links wurde versucht, Buchstaben einzugeben. Dies entspricht nicht dem definierten Pattern. Auch das Bild rechts zeigt die Abweisung eines Eingabeversuches: Obwohl hier tatsächlich nur Ziffern verwendet werden, entspricht die Eingabe nicht der geforderten Stellenzahl (5).

Eingabe mit vier oder fünf Stellen

Das Formular wäre mit der strengen Vorgabe einer fünfstelligen Postleitzahl für österreichische Adressen sehr ungünstig gestaltet. Eine kleine Änderung des in geschweiften Klammern formulierten Wertes ermöglicht jedoch auch dies und bietet die Wahlfreiheit einer vier- oder fünfstelligen Eingabe. Die zulässigen Werte werden durch ein Komma voneinander getrennt.

```
<input type="text" name="plz" id="plz" pattern="[0-9]{4,5}"
placeholder="12345">
```

Genau genommen wird hier ein Bereich festgelegt, der im Sinne „von ... bis“ gelesen werden muss. Wird beispielsweise das Wertepaar „2,5“ in die geschweiften Klammern geschrieben, dann sind zwei, drei, vier- und

fünfstellige Eingaben gültig. Einstellige Eingaben und Zahlen mit mehr als fünf Stellen werden dagegen zurück gewiesen.

Gemischte Eingabe mit definiertem Format

Im nächsten Beispiel sollen in einem Text-Eingabefeld österreichische Kfz-Nummern erfasst werden³¹. Hierbei sollen jedoch nur Standard-Formate akzeptiert werden (keine Wahlkennzeichen). Eine sehr grobe Validierung der Eingabe ist mit dem folgenden Pattern möglich:

```
pattern=" [A-Z] {1,2} - [0-9] {1,5} [A-Z] {1,3} "
```

- Der rot geschriebene Teil schränkt die Eingabe des ersten bzw. ersten und zweiten Zeichens auf große Buchstaben ein. Ziffern sind hier nicht erlaubt.
- Es folgt ein Bindestrich als Trennzeichen. Dieses wird in der einzugebenden Zeichenkette erwartet. Fehlt es, wird die Eingabe zurück gewiesen.
- Nun sollen – dargestellt im grünen Textteil – Ziffern eingegeben werden. Es muss mindestens eine Ziffer eingegeben werden, maximal darf die Zahl hier fünfstellig sein.
- Zum Abschluss werden wieder reine Buchstaben erwartet: Es muss mindestens ein Buchstabe eingegeben werden, jedoch dürfen maximal drei Buchstaben an dieser Stelle Teil des Kennzeichens sein.

Die folgenden Abbildungen zeigen die Wirkung dieser Pattern-Definition am Beispiel verschiedener Eingabeversuche:

³¹ Die hier gezeigten Beispiele erfassen nicht alle gültigen Vorgaben zur Gestaltung von Vormerkzeichen in Österreich. Sie dienen lediglich der Erläuterung des „Pattern“-Attributes in einem HTML-Eingabe-Formular.

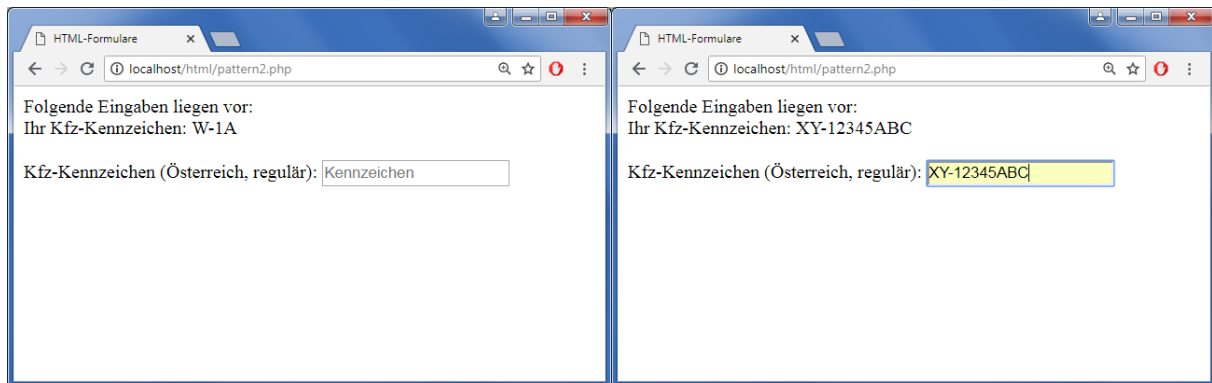


Abbildung 62: In beiden Fällen entsprechen die Eingaben den im Pattern-Attribut gemachten Vorgaben.

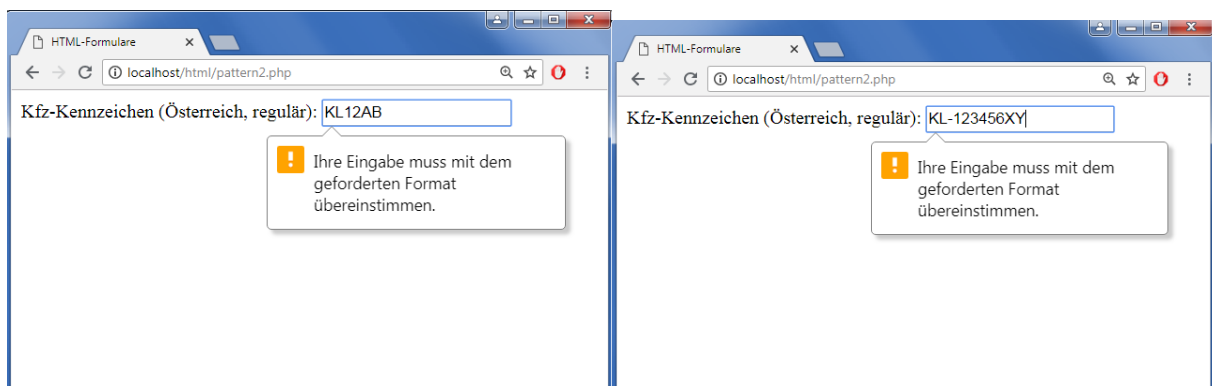


Abbildung 63: Die Eingabe im Bild links wird zurück gewiesen, weil der geforderte Bindestrich fehlt. Im Bild rechts wurden zu viele Stellen im Ziffernblock eingegeben.

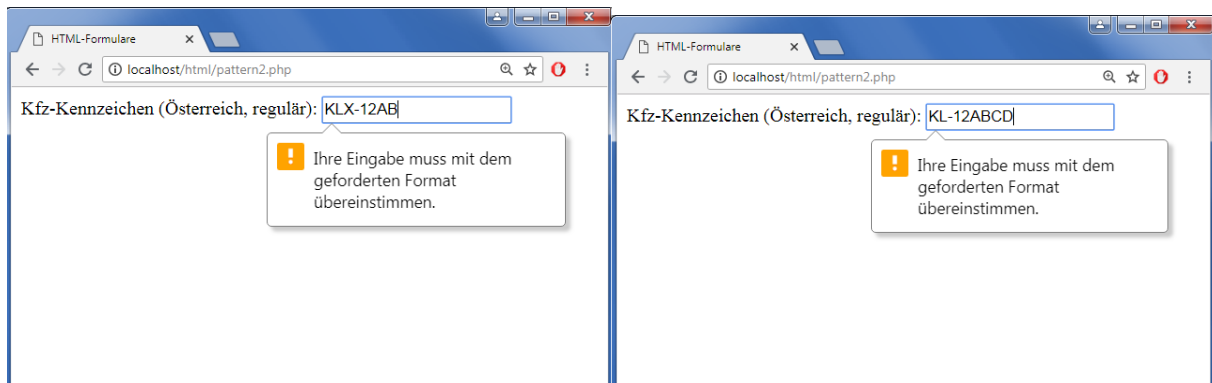


Abbildung 64: Der erste Buchstabenblock darf nur zwei Stellen umfassen (Bild links). Im rechten Bild wurde der letzte Buchstabenblock zu lang formuliert. Er besitzt hier vier Zeichen (erlaubt sind drei).

Groß und Kleinschreibung ohne Ziffern

Im folgenden Beispiel sollen für die Eingabe ausschließlich Buchstaben zugelassen werden. Dabei spielt es jedoch keine Rolle, ob diese in Groß- oder Kleinschrift eingegeben werden. Zudem besteht keine Begrenzung der

zulässigen Zeichenzahl, was durch das Pluszeichen hinter der eckigen Klammer (grün hervor gehoben) deklariert wird.

Es gibt zwei Möglichkeiten, gemischte Groß- und Kleinschreibung im Text zu erreichen:

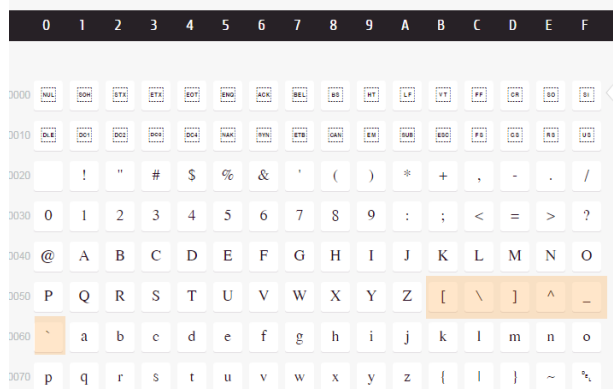
[A-Za-z] und [A-z].

```
<input type="text" name="test" id="test" pattern="[A-Za-z]+"
placeholder="Texteingabe">
```

Alternativ kann auch folgendes Pattern verwendet werden:

```
<input type="text" name="test" id="test" pattern="[A-z]+"
placeholder="Texteingabe">
```

Auch dieses lässt die Eingabe von Buchstaben – wahlweise in Groß- oder Kleinschreibung – zu. Hier ist jedoch zu berücksichtigen, wie das Pattern interpretiert wird. Grundlage ist die *Unicode-Tabelle* und die darin definierte Codierung der Zeichen. Bedauerlicherweise sind zwischen den Groß- und Kleinbuchstaben weitere Zeichen definiert, die bei einer Pattern-Deklaration [A-Za-z] **NICHT** Teil des Patterns sind.



	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	NAI	SON	STX	ETX	SOI	SHI	ACH	BEL	BS	HT	LP	VT	FF	CR	SO	SI
0010	DL	DC	DC	DC	DC	DC	DC	DC	DC	DC	DC	DC	DC	DC	DC	DC
0020		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
0030	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
0040	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
0050	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
0060	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
0070	p	q	r	s	t	u	v	w	x	y	z	{		}	~	?

Abbildung 65: Zwischen den großen und kleinen Buchstaben sieht die Unicode-Tabelle weitere Zeichen, u.a. "Underline" vor. Dies ist bei der Deklaration eines Pattern zu berücksichtigen. Quelle: <https://unicode-table.com> (Auszug)

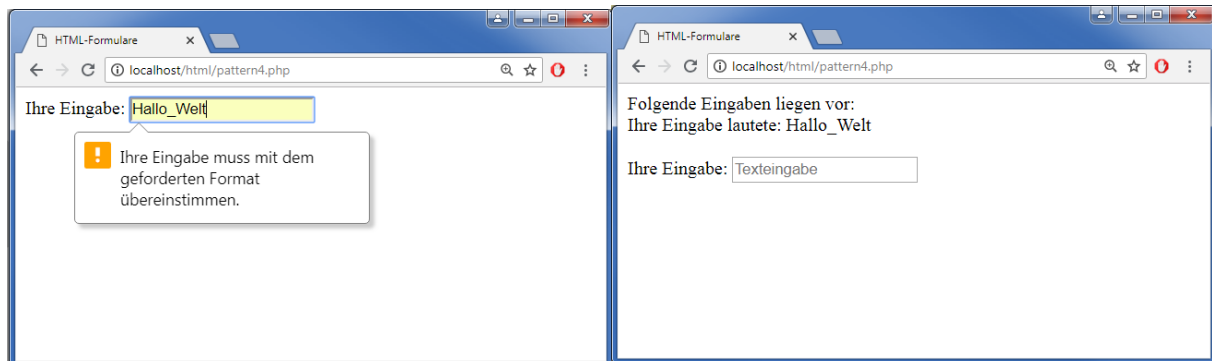


Abbildung 66: Deklaration des Pattern im Bild links: `[A-Za-z]+` - im Bild rechts: `[A-z]+`

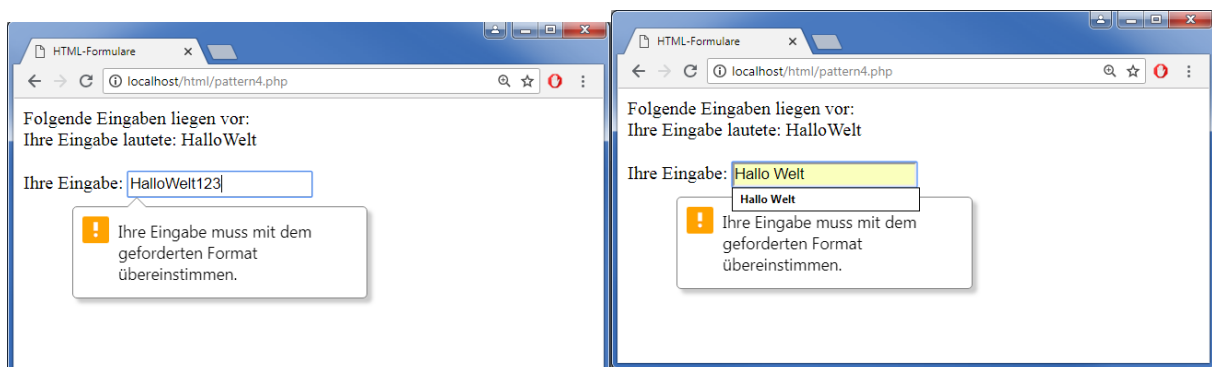


Abbildung 67: Leerzeichen gehören nicht zu den Buchstaben! Aus diesem Grund ist die Eingabe im rechten Bild ungültig. Im linken Bild wurde versucht, neben Buchstaben auch Ziffern zu verwenden. Das ist mit dem gegebenen Pattern ausgeschlossen worden.

Leerzeichen akzeptieren

Das Leerzeichen gehört nicht zu den Buchstaben oder Ziffern. Deswegen muss es in maskierter Codierung deklariert werden. Hier lautet der Code: Backslash (`\`), ergänzt um ein kleines „s“. Das „s“ steht für „Space“.

```
pattern="[A-Za-z\s]+"
```

Doch Vorsicht: Wird das „s“ groß geschrieben, wird das Leerzeichen ausdrücklich ausgeschlossen!

`pattern="[A-Za-z\S]+"` ist nicht gleich `pattern="[A-Za-z\s]+"` !!!

Es gibt weitere Codierungen, auf die an dieser Stelle jedoch nicht weiter vertiefend eingegangen werden soll. Beispielsweise steht der Code `\d` für „digit“ (Ziffern). `/D` schließt die Verwendung von Ziffern aus³².

³² Ausschlüsse sind in Pattern i.d.R. überflüssig, da nur die Zeichen gültig sind, die ausschließlich als zulässig deklariert werden.

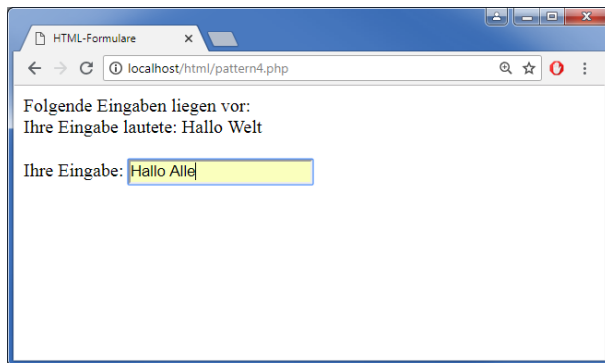


Abbildung 68: Neben Buchstaben (Groß- und Kleinschreibung) ist auch das Leerzeichen an dieser Stelle erlaubt.

Direkter Textvergleich

Mit dem Attribut „pattern“ lassen sich auch Referenztexte³³ definieren, der bei der Eingabe unbedingt enthalten sein muss.

Im ersten Beispiel wird der Name „Hugo“ als Wert des Pattern-Argumentes gesetzt. Dieser muss exakt in der vorgegebenen Weise (Groß-/Kleinschreibung ist zu beachten!) eingegeben werden.

```
<input type="text" name="user" id="user" pattern="Hugo"
placeholder="Username">
```

Das Beispiel soll nun etwas erweitert werden: Es soll die Zeichenkette im Pattern Teil der Eingabe sein. Der Name „Hugo“ muss also an irgendeiner Stelle innerhalb der Zeichenkette auftauchen. Das wird durch den Punkt (ein beliebiges Zeichen) in Kombination mit dem Pluszeichen (beliebig viele Zeichen) erreicht, die hier vor und nach dem Namen gesetzt werden.

```
<input type="text" name="user" id="user" pattern=".+Hugo.+"
placeholder="Username">
```

Das dritte Beispiel soll Alternativen berücksichtigen. So können beispielsweise einige Namen in verschiedener Schreibweise dargestellt werden. Hier der Name „Maier“ bzw. „Meier“. Der Punkt im Pattern drückt wieder ein beliebiges Zeichen aus, was an dieser Stelle in der eingegebenen Zeichenkette gesetzt werden kann.

```
<input type="text" name="user" id="user" pattern="M.ier"
placeholder="Username">
```

³³ Wichtig: Pattern, die eine Eingabe auf bestimmte Texte begrenzen, sind selbstverständlich kein Ersatz für eine Zugangssicherung! Die Pattern sind im Quellcode im Klartext lesbar.

Problematisch am zuvor gezeigten Beispiel ist allerdings, dass die Eingabebeschränkung nicht exakt ist. Neben den zulässigen Schreibweisen für den Namen („Maier“ oder „Meier“) könnte auch ein Schreibfehler akzeptiert werden (z.B.: „Mmier“). Das wäre ärgerlich, denn diese Eingabe würde zwar vom HTML-Formular akzeptiert, jedoch in einer später durchsuchten Datenbank nicht gefunden werden.

Abhilfe schafft die Vorgabe der richtigen Schreibweisen im Pattern, wie es das folgende Beispiel zeigt. Das Zeichen „Pipe“ (|) steht für ODER bzw. OR. Akzeptiert werden hier also die Eingaben „Meier“ ODER „Maier“. Andere Schreibweisen werden abgewiesen.

```
<input type="text" name="user" id="user" pattern="Meier|Maier"
placeholder="Username">
```

Protokoll-Präfix vor Webadresse

Die eben beschriebenen Pattern können auch für weitere Eingabebegrenzungen bzw. Eingabeunterstützungen genutzt werden. Neben dem Typ „url“ eines <input>-Elementes kann die korrekte Eingabe (inkl. des Protokollpräfixes) einer Webadresse auch mit einem reinen Text-Typ erzwungen werden. Das Protokollpräfix ist entweder http:// (unverschlüsseltes Hypertext Transfer Protocol) oder https:// (verschlüsseltes Protokoll, also Hypertext Transfer Protocol Secure).

Um das entscheidende Zeichen „s“ optional verwenden zu können, wird diesem Zeichen im Pattern ein Fragezeichen nachgestellt. Natürlich ist eine Webseite bedeutend länger als das Protokollpräfix. Deswegen muss das Pattern auch Eingaben weiterer Zeichen erlauben. Neben dem „Punkt“, der für ein beliebiges Zeichen steht, erlaubt das Pluszeichen eine nicht begrenzte Zahl von Zeichen insgesamt. Nur in dieser Kombination kann eine vollständige Adresse eingetragen werden.

```
<input type="text" name="user" id="user" pattern="https?: /\.+"
placeholder="Webseite">
```

Pflichteingaben

Zum Abschluss in die Einführung der HTML-Formulare soll ein wichtiges Attribut vorgestellt werden, das in jedem <input>- und <textarea>-Feld

verwendet werden kann: „**required**“. Ist dieses Attribut in einem Eingabeelement vorhanden, dann kann das gesamte Formular erst abgeschickt werden, sobald in diesem Feld eine Eingabe gemacht wurde. Leere Felder werden nicht akzeptiert!

```
<input type="number" name="wahl" required>
```



Abbildung 69: Der Versuch, das Formular zu senden, ohne dieses Feld auszufüllen scheitert, weil im <input>-Element das Attribut "required" verwendet wurde.